4. System Analysis & Design

Data source: MIMIC-IV (de-identified clinical records, PhysioNet)

Introduction

This document is the first of six deliverables for the **Smart Medical Diagnostics & Risk Scoring System**. It defines the problem, the people who use the system, what the system must do, and how it is built at a high level. Later documents cover the database (section 2), data flow and behaviour (section 3), the screens (section 4), deployment (section 5) and interfaces, testing and rollout (section 6).

The system helps emergency staff see which patients may be at higher risk. It works on 222 emergency triage records, 100 patients, 275 hospital admissions and about 108,000 lab results from MIMIC-IV, a de-identified hospital database. It is a **data pipeline plus reports**: Python reads and scores the data, SQL Server stores it, a machine learning model adds a probability, and Power BI shows the result. There is no website, mobile app or web service.

Term	Meaning
Triage	The first assessment in the emergency department, which decides who is seen first.
Acuity (ESI)	Urgency level from 1 to 5 given by the triage nurse. 1 is most urgent.
Vital signs	Heart rate, breathing rate, oxygen level, blood pressure and temperature.
Risk score and tier	A number worked out from the vital signs, grouped into High, Medium or Low. Unscored means too little data.
Report	The Power BI screens that users look at.
Database / warehouse	Where the data is stored (SQL Server). The warehouse is the processed part, schema dw.
Pipeline	The chain of Python programs that read, clean, score and store the data (ETL: extract, transform, load).
Random Forest	A machine learning method that combines many decision trees to give a probability.
De-identified	Names and direct identifiers have been removed from the records.

1. System Analysis

1. Problem Statement

In an emergency department, the triage nurse records vital signs and an urgency level for every arrival. The records are stored as separate tables, and the same patient also has admissions and thousands of lab results in other tables. Looking at these files one by one it is hard to answer simple questions quickly: *who needs attention first, what are this patient's abnormal results, and how busy is the hospital?*

The raw data also has quality problems that make direct use risky:

- **Impossible values.** One triage record has a diastolic blood pressure of 879, and one temperature (36.5) is in Celsius while the rest are in Fahrenheit.
- **Missing vital signs.** 23 of the 222 triage records have no vital signs at all. A missing value must never be read as a healthy value.
- **Inconsistent fields.** The pain column mixes numbers (some above 10) with text such as *unable*; 20 admissions have language '?'.
- **Weak links.** Triage records have no date and no admission number, and 26.4% of lab results (28,420 of 107,727) have no admission number.

> **Problem in one sentence**
> The project builds a repeatable pipeline that cleans the data, scores each triage record for risk, adds a model-based probability, stores everything in a warehouse and shows it in three Power BI screens: Triage, Patient and Executive.

1.2 Project Objectives

#	Objective	How it is measured
O1	Load and check all five source files without manual steps.	One command runs end to end; row counts match the source (222 / 100 / 275 / 107,727 / 1,622).
O2	Clean impossible and inconsistent values and keep a record of every change.	Each change appears in the activity log with old and new value.
O3	Give every triage record a risk score and tier from its vital signs.	Tiers: 3 High, 17 Medium, 179 Low, 23 Unscored (sums to 222). 10 hand-checked records match.
O4	Never show a record with too little data as low risk.	All 23 records with fewer than 3 scored vital signs are Unscored.
O5	Add a probability from a machine learning model.	Random Forest trained and applied; accuracy and AUC stored with each run.
O6	Store results in a SQL Server warehouse that the reports read.	Schema dw holds dimensions, facts, logs and three views.
O7	Show three report pages that answer the main questions.	Seven acceptance scenarios (S1 to S7) pass.
O8	Protect patient privacy.	Only a masked ID is shown on screen; raw tables are never modified.

1.3 Scope

In scope	Out of scope
Batch pipeline in Python (pandas, NumPy) for the five MIMIC-IV files	Live data feeds from a hospital system
SQL Server data warehouse: raw schemas ed, hosp and results schema dw	A website, mobile app or web service (API)
NEWS2-style risk score and tiers from five vital signs	Clinical decisions or diagnosis; the score supports a reviewer, it does not replace one

In scope	Out of scope
Random Forest probability (scikit-learn)	Real patient data; only the de-identified MIMIC-IV subset is used
Power BI report with three pages (Tableau as an alternative front end)	Staffing, bed numbers and capacity; the data does not contain them

1.4 Stakeholders and Actors

Actor	Role in the system	Main interest
Triage nurse	Reads the Triage page	Who should be seen first, and why.
Physician / analyst	Reads the Patient page	A patient's abnormal labs and lab trends; triage history.
Operations manager	Reads the Executive page	Load by hour, length of stay, admission types, deaths.
Data engineer	Runs and maintains the pipeline	Clean runs, readable logs, matching row counts.
Supervisor / evaluator	Reviews the finished report (UAT)	Scenarios pass and open issues are agreed.
MIMIC-IV dataset (external)	Supplies the five CSV files	Files arrive in the expected format.

1.5 Understanding the Data

The design is driven by what the files actually contain. Dates in MIMIC-IV are shifted by the data provider, so years such as 2185 are not real.

File	Rows	What it holds	Key facts found
ed / triage	222	One record per emergency stay: vital signs, pain, acuity, main complaint.	64 of 100 patients appear. Acuity: 18 at level 1, 97 at 2, 90 at 3, 2 at 4, 15 missing. No date, no admission number.
hosp / patients	100	Gender, age, anchor year, date of death.	57 male, 43 female; ages 21 to 91 (mean 61.8); 31 have a date of death.
hosp / admissions	275	Admission type, times, discharge place, death flag, ED times.	9 admission types; 15 in-hospital deaths (5.5%); median stay 4.9 days; ED times for 182 admissions.
hosp / labevents	107,727	One lab result per row with value, unit, normal range, abnormal flag.	11,122 specimens; 498 different tests; 37.4% flagged abnormal; 73.6% linked to an admission.
hosp / d_labitems	1,622	Lab test names, fluid and category.	3 categories (Hematology, Chemistry, Blood Gas); 3 missing labels; fluid codes 'Q', 'I', 'Fluid' look like typing errors.

Findings that shape the design

Finding in the data	Design response
Diastolic pressure 879; temperature 36.5 in Celsius; pain 13 and 'unable'.	A cleaning step with ranges in a settings file. Bad values become missing, units are converted, every change is logged.
23 triage records have no vital signs; others have some missing.	A record is scored only if at least 3 of the 5 vital signs are present. Otherwise it is Unscored and is shown near the top, not as Low.
Triage has no date and no admission number.	Triage is linked to patients only. Waiting time cannot be shown; the Triage page orders by risk.
26.4% of lab results have no admission number.	The admission key in the lab fact table is optional. Patient charts use the lab date and time.
The lab 'flag' column holds only 'abnormal' or empty.	Stored as a yes/no field is_abnormal; the Patient page also shows the normal range.
2,893 lab values are text (for example NEG, HOLD) rather than numbers.	Both value_num and value_text are kept; charts use the numeric value only.
Only 15 of 275 admissions end in death, and 31 patients have a death date.	In-hospital death comes from the admission flag, not from the patient date of death.
Only 222 triage records; 207 have an acuity level.	The model is validated with cross-validation and its limits are stated; the score does not depend on the model.

1.6 Use Case Diagram

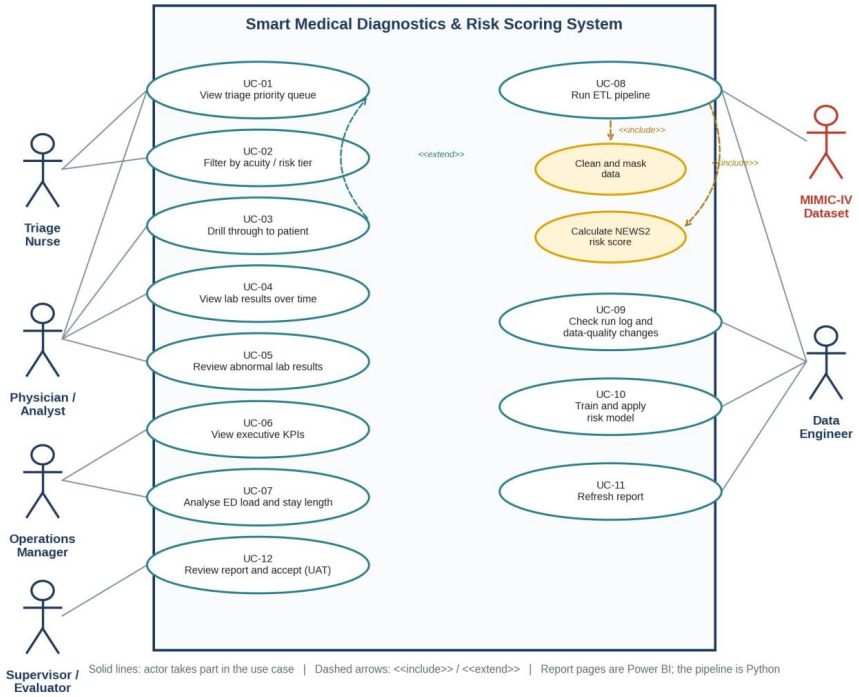


Figure 1.1. Use case diagram. Actors on the left read the report; the data engineer and the dataset on the right feed it.

1.6.1 Use case summary

ID	Use case	Actor(s)	Result
UC-01	View triage priority queue	Triage nurse, physician	Records ordered by risk with tier and score.
UC-02	Filter by acuity or risk tier	Triage nurse	Cards, queue and chart change together.
UC-03	Drill through to patient	Physician	Patient page opens for the selected record.
UC-04	View lab results over time	Physician	Line charts with normal range shaded.
UC-05	Review abnormal lab results	Physician	Latest abnormal result per test.
UC-06	View executive KPIs	Operations manager	Admissions, ED registrations, median stay, deaths.
UC-07	Analyse ED load and stay length	Operations manager	Hourly, stay-band and admission-type charts.
UC-08	Run ETL pipeline (includes clean and mask data, calculate NEWS2 score)	Data engineer, dataset	Warehouse tables loaded.
UC-09	Check run log and data-quality changes	Data engineer	Log lines and run summary.
UC-10	Train and apply risk model	Data engineer	Probability per triage record, metrics saved.
UC-11	Refresh report	Data engineer	Report shows the new run.
UC-12	Review report and accept (UAT)	Supervisor / evaluator	Scenarios S1 to S7 recorded.

1.6.2 Use case descriptions

UC-01 View triage priority queue

Actor	Triage nurse (also physician)
Precondition	The pipeline has finished at least one run; the report is connected to the warehouse.
Main flow	1. The nurse opens the Triage page. 2. The report reads view vw_triage_queue. 3. Four cards show the count in each tier. 4. The queue lists records: High first, then Unscored, then Medium, then Low. 5. Each row shows tier, score, stay ID, complaint, vital signs and acuity.
Alternative flow	A record with no vital signs is shown with a dash in every vital sign and the tier Unscored (grey). It is never shown as Low.
Postcondition	The nurse knows which record to look at first and why.

UC-03 Drill through to patient

Actor	Physician
Precondition	UC-01 completed; the queue shows at least one row.
Main flow	1. The physician selects a queue row. 2. The report opens the Patient page for that patient. 3. The page shows the profile strip, two lab line charts, the abnormal-labs table and the patient's triage records.
Alternative flow	If the patient has no lab results the charts are empty and the page says so; the profile and triage records still show.
Postcondition	The Patient page is filtered to one patient.

UC-04 View lab results over time

Actor	Physician / analyst
Precondition	A patient is selected; lab results exist for the chosen test.
Main flow	1. The user picks a lab test in the slicer. 2. The line chart plots value against time. 3. A shaded band marks the normal range; points outside it are highlighted.

Alternative flow	Results that are text (for example NEG) are not plotted; they appear only in the table.
Postcondition	The user sees whether a result is moving out of range.

UC-06 View executive KPIs

Actor	Operations manager
Precondition	The warehouse holds all 275 admissions.
Main flow	1. The manager opens the Executive page. 2. Cards show admissions (275), ED registrations (182), median stay (4.9 days) and in-hospital deaths (15, 5.5%). 3. Charts show registrations by hour, length of stay and admission type. 4. The manager can filter by admission type.
Alternative flow	ED registration time exists for 182 of 275 admissions only; the hourly chart covers those and says so.
Postcondition	The manager sees demand and outcomes. Capacity is not shown: the data has no staffing or beds.

UC-08 Run ETL pipeline

Actor	Data engineer; MIMIC-IV dataset supplies the files
Precondition	SQL Server is running; the five CSV files are in the data folder; the settings file has the server name and path.
Main flow	1. The engineer starts the pipeline with one command. 2. The pipeline reads and checks the files. 3. It cleans the data and logs each change. 4. It prepares measurements and calculates the risk score and tier. 5. It trains and applies the Random Forest model. 6. It saves results to schema dw and checks the row counts. 7. It prints a run summary.
Alternative flow	If an expected column is missing, the pipeline stops with a message and writes nothing. If row counts do not match after saving, the pipeline reports the difference and exits with an error.
Postcondition	Warehouse tables and the run log are up to date; raw tables are unchanged.

UC-10 Train and apply risk model

Actor	Data engineer
Precondition	Clean measurements exist for the triage records.
Main flow	1. The pipeline builds the feature table. 2. It trains a Random Forest classifier with cross-validation. 3. It saves accuracy and AUC to dw.model_run. 4. It writes a probability for each record to dw.fact_triage_scored.
Alternative flow	Records without enough measurements get no probability (null) and stay Unscored.
Postcondition	Each scoreable record has a probability next to its risk tier.

1.7 Functional Requirements

Priority uses MoSCoW: **M** must have, **S** should have.

ID	Requirement	Pri.
FR-01	Read the five CSV files and confirm the expected columns; stop with a clear message if one is missing.	M
FR-02	Load the files unchanged into raw schemas ed and hosp.	M
FR-03	Clean the data: convert Celsius to Fahrenheit, set impossible vital signs and pain values to missing, standardise categories (for example language '?' becomes Unknown).	M
FR-04	Write every change made to the data to the activity log (table, key, old value, new value, rule).	M
FR-05	Keep the real patient ID private; show only a masked ID (for example ***810) in reports and logs.	M
FR-06	Calculate extra measurements: shock index (heart rate / systolic pressure) and mean arterial pressure.	S
FR-07	Give points to each of five vital signs (heart rate, breathing rate, oxygen, systolic pressure, temperature) from bands held in the settings file.	M
FR-08	Set the tier: High (total 7 or more); Medium (5 or 6, or any single vital sign at the worst level); Low (otherwise); Unscored (fewer than 3 vital signs recorded).	M
FR-09	Train a Random Forest classifier, store accuracy and AUC, and store a probability for each scoreable record.	S
FR-10	Write results to schema dw (dimensions, facts, logs) and create views vw_triage_queue, vw_patient_timeline, vw_exec_kpis.	M
FR-11	After saving, compare row counts with the source and report any difference.	M
FR-12	Running the pipeline twice gives the same row counts (repeatable).	M
FR-13	Triage page: four tier cards, a priority queue (High, Unscored, Medium, Low) and a tier chart.	M
FR-14	Slicers for acuity (ESI) and risk tier update all visuals together.	M
FR-15	Drill-through from a queue row to the Patient page.	M
FR-16	Patient page: profile strip, lab line charts with shaded normal range, latest abnormal result per test, patient's triage records.	M
FR-17	Executive page: admissions, ED registrations, median stay, in-hospital deaths; charts by hour, stay length and admission type.	M
FR-18	Show missing values as a dash; never show an Unscored record as Low.	M
FR-19	Page tabs on every page for moving between Triage, Patient and Executive.	M

1.8 Non-Functional Requirements

Area	Requirement	Target / check
Accuracy	Scores calculated by the pipeline equal scores worked out by hand.	10 randomly chosen records match (check V2).
Reliability	A failed run changes nothing in dw; a repeat run gives the same counts.	Test I3; stop on structural error.
Performance	Full run on one workstation; report pages open without waiting.	Target: pipeline under 10 minutes for 107,727 lab rows; page refresh under 5 seconds.
Privacy	Only de-identified data; masked ID on screen and in logs; raw tables never modified.	Test U1; review of views.
Security	Windows authentication to SQL Server; no password stored in files.	Settings file contains server name and paths only.
Auditability	Every change to the data and every run is recorded.	Activity log and dw.etl_run_log.
Usability	Most urgent information top-left; at most six visuals per page; same layout on all pages.	Scenarios S1 to S7.

Area	Requirement	Target / check
Accessibility	Colour is never the only signal; text contrast at least 4.5 : 1; each chart has alt text.	Palette check; greyscale review (S7).
Maintainability	Code split into small classes; limits and bands in a settings file; automated tests.	pytest suite; no numbers hard-coded.
Portability	Runs on one Windows computer with Python 3, SQL Server and Power BI Desktop.	Deployment steps in section 5 and 6.
Scalability	Not built; the document says what a larger extract would need.	Section 6.3.

1.9 Assumptions and Constraints

- The data is a fixed MIMIC-IV subset; there is no live feed.
- Triage records cannot be tied to one admission, so labs and admissions do not feed the risk score.
- Only five vital signs are available. The score therefore leaves out consciousness and oxygen-therapy items that the full NEWS2 chart includes.
- The score bands are a draft agreed by the team. They reproduce the tier counts in this document (3 / 17 / 179 / 23) and can be changed in the settings file.
- **To confirm with the supervisors:** the machine learning target. This document assumes it predicts an urgent acuity (ESI 1 or 2) from the vital signs, because hospital outcomes cannot be linked to triage records.
- Dates are shifted by MIMIC-IV, so no calendar conclusions (seasons, weekdays) are drawn.

1.10 Software Architecture

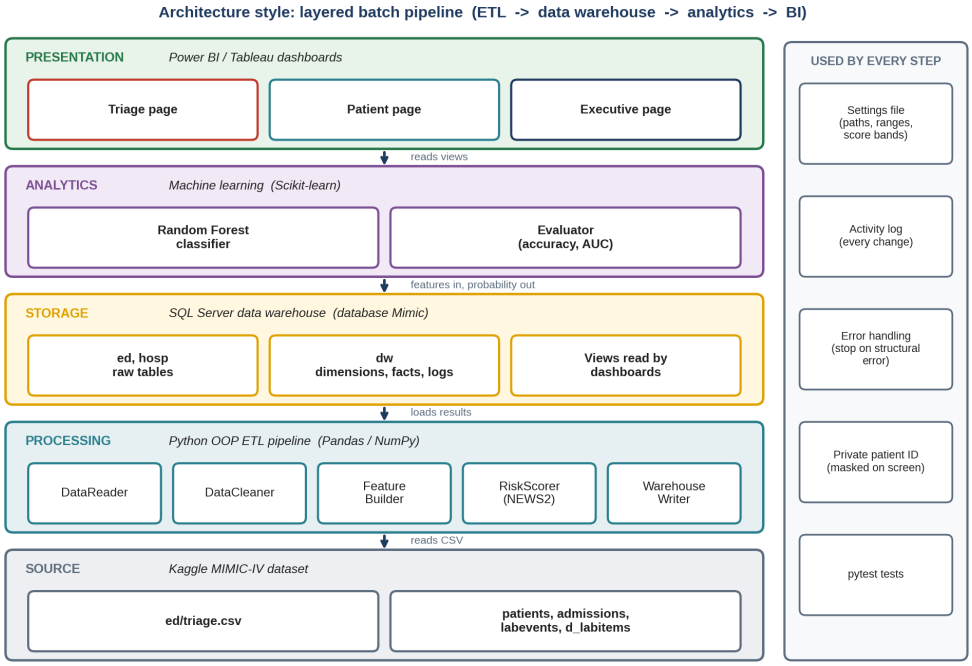


Figure 1.2. High-level architecture: a layered batch pipeline from source files to dashboards.

The system is a **layered batch pipeline**. Data moves in one direction, from the source files to the dashboards, and each layer has one job. This is the right style because the data does not change during the day, users only read results, and every number must be repeatable and traceable.



Figure 1.3. The five stages of the system.

Layer	Technology	Responsibility
Source	Kaggle MIMIC-IV dataset (CSV)	Provides ed/triage and hosp/patients, admissions, labevents, d_labitems.
Processing	Python OOP ETL pipeline: pandas, NumPy, logging, pytest	Reads and checks, cleans, builds measurements, scores risk, writes to the warehouse.
Storage	SQL Server data warehouse, database Mimic	Raw schemas ed, hosp; results schema dw; views for the report.
Analytics	Scikit-learn Random Forest	Learns from the measurements and writes a probability per record.
Presentation	Power BI Desktop (Tableau as alternative)	Triage, Patient and Executive pages read the views.

Architecture decisions

Decision	Reason	Alternative considered
Batch pipeline instead of an application	Data is static; users only need to read results. No server or login is needed.	Web application with a service layer: more to build and maintain with no extra benefit here.
Object-oriented Python modules	One class per step (reader, cleaner, scorer, model, writer) makes testing and change easy.	A single long script.
SQL Server warehouse with raw and result schemas	Raw data stays untouched; results can be rebuilt at any time; Power BI reads views.	Reading CSV files directly in Power BI: no history, no logs, harder to test.
Score first, model second	The NEWS2-style score is transparent and works without training; the model adds a probability.	Model only: harder to explain to clinicians.
Unscored tier	Missing data is unknown, not safe.	Treating missing values as normal.

How the classic layers map to this system

Because there is no interactive application, the usual web layers do not apply. The closest idea is: **Model** = the warehouse (dw), **View** = the Power BI pages, and **Controller** = the Python pipeline that prepares and loads what the views show. Cross-cutting parts used by every step are the settings file, the activity log, error handling and the private patient ID.

2. Database Design & Data Modeling

The database is a SQL Server database named **Mimic** with three kinds of schema. **ed** and **hosp** hold the source files exactly as supplied (raw). **dw** holds the processed results (the data warehouse) that Power BI reads through views. Raw tables are never changed: every cleaning step happens in Python and the result goes to dw.

Schema	Contents	Written by	Read by
ed	triage	Pipeline: load step	Pipeline
hosp	patients, admissions, labevents, d_labitems	Pipeline: load step	Pipeline
dw	3 dimensions, 3 fact tables, 3 log tables, 3 views	Pipeline: save step; model step (probability)	Power BI / Tableau (views only), model step

2.1 Source Data Profile

Table	Rows	Columns	Primary key	Foreign keys	Missing values worth noting
ed.triage	222	11	stay_id	subject_id	23 rows with no vital signs; acuity missing 15; DBP 879 (1 row); temperature 36.5 in Celsius (1 row)
hosp.patients	100	6	subject_id	-	dod present for 31 patients
hosp.admissions	275	16	hadm_id	subject_id	discharge_location missing 42; marital status missing 12; ED times missing 93; language '?' 20
hosp.labevents	107,727	16	labevent_id	subject_id, hadm_id, itemid	hadm_id missing 28,420 (26.4%); value text missing 9,588; numeric value missing 12,481; unit missing 16,203; reference range missing 18,728
hosp.d_labitems	1,622	5	itemid	-	3 labels missing; 498 of 1,622 tests are used in labevents

Referential integrity of the source is clean: every triage and admission *subject_id* exists in patients (222 of 222 and 275 of 275), every lab *itemid* exists in d_labitems (107,727 of 107,727) and every non-empty lab *hadm_id* exists in admissions (79,307 of 79,307).

2.2 Entity-Relationship Diagram: Source Tables

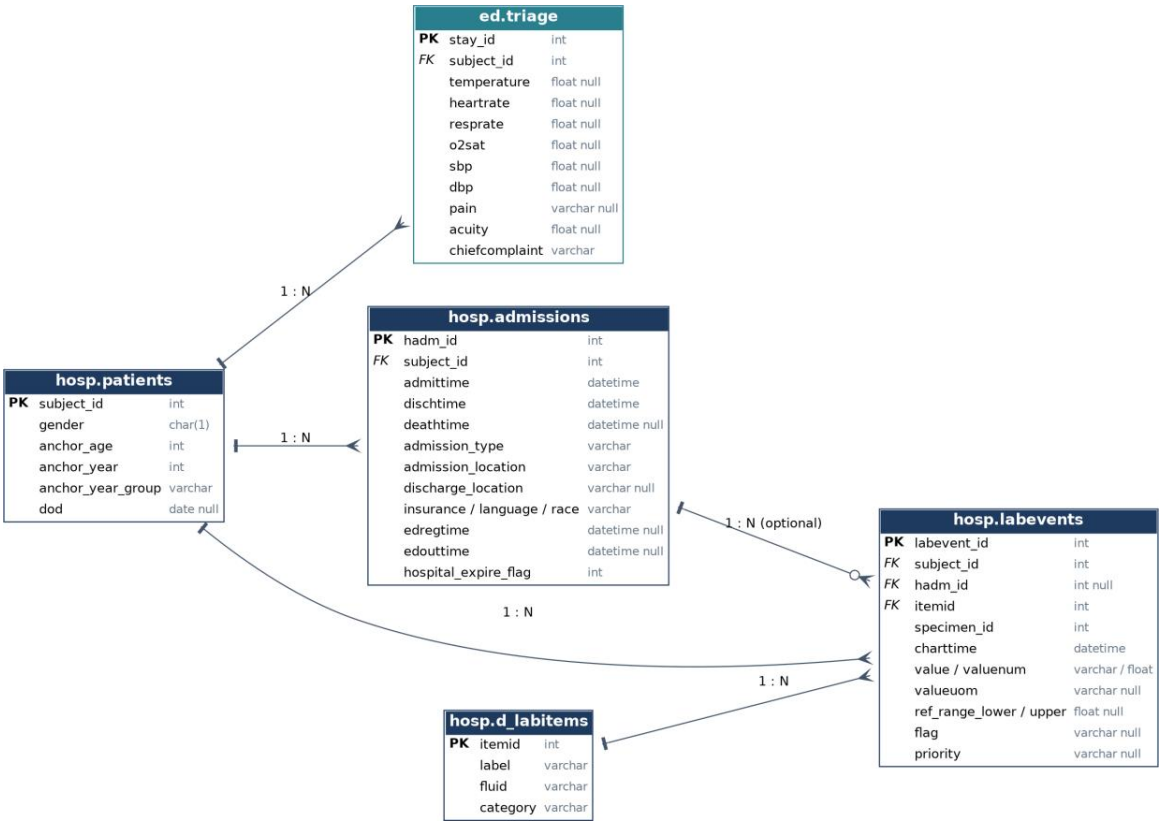


Figure 2.1. ERD of the five source tables (schemas ed and hosp).

Relationship	Cardinality	Comment
patients - triage	1 : N	A patient can have several triage stays (up to 23 in the data). 36 patients have none.
patients - admissions	1 : N	Every patient has at least one admission; the most is 20.
patients - labevents	1 : N	Every lab result belongs to a patient.
admissions - labevents	1 : N (optional)	73.6% of lab results have an admission; the rest have none.
d_labitems - labevents	1 : N	Each result belongs to one lab test; 498 tests are used.
triage - admissions	no key	Not linkable: triage has no date and no hadm_id. The design does not invent a link.

2.3 Entity-Relationship Diagram: Data Warehouse (schema dw)

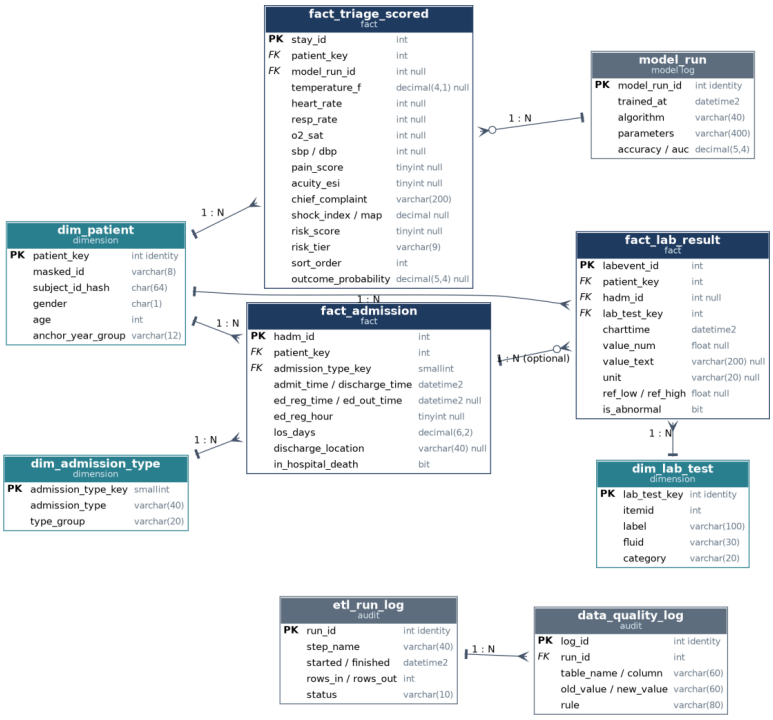


Figure 2.2. ERD of the warehouse: a star of three dimensions and three fact tables, plus model and audit tables.

The warehouse is a **star schema**. Facts hold the numbers that the report counts and plots; dimensions describe who or what the fact is about. A star keeps the report's queries simple (one join from a fact to each dimension) and is the usual shape for Power BI and Tableau.

Table	Type	Grain (one row is...)	Rows expected
dim_patient	Dimension	one patient	100
dim_admission_type	Dimension	one admission type	9
dim_lab_test	Dimension	one lab test (itemid)	1,622 (498 used)
fact_triage_scored	Fact	one emergency triage stay, scored	222
fact_admission	Fact	one hospital admission	275
fact_lab_result	Fact	one lab result	107,727
model_run	Model log	one training run of the Random Forest	1 per run
etl_run_log	Audit	one pipeline step in one run	about 7 per run
data_quality_log	Audit	one change made during cleaning	few per run

2.4 Logical Schema

Logical design lists every table with its attributes, keys and meaning, independent of the SQL Server syntax used later.

dw.dim_patient

Column	Type	Key	Null	Notes
patient_key	int identity	PK	No	Surrogate key used by every fact table.
masked_id	varchar(8)		No	For example ***810; the only patient identifier shown on screen.
subject_id_hash	char(64)		No	SHA-256 of subject_id, so the real ID is never stored in dw.
gender	char(1)		No	M or F.
age	int		No	anchor_age.
anchor_year_group	varchar(12)		No	For example 2014 - 2016.

dw.dim_admission_type

Column	Type	Key	Null	Notes
admission_type_key	smallint	PK	No	
admission_type	varchar(40)		No	EW EMER., URGENT, OBSERVATION ADMIT, ... (9 values).
type_group	varchar(20)		No	Emergency, Observation, Elective or Surgical.

dw.dim_lab_test

Column	Type	Key	Null	Notes
lab_test_key	int identity	PK	No	
itemid	int		No	Source test code, unique.
label	varchar(100)		No	Missing labels become 'Unknown test '.
fluid	varchar(30)		No	Blood, Urine, ...; codes 'Q', 'I', 'Fluid' mapped to Other.
category	varchar(20)		No	Hematology, Chemistry or Blood Gas.

dw.fact_triage_scored

Column	Type	Key	Null	Notes
stay_id	int	PK	No	Source stay number.
patient_key	int	FK	No	-> dim_patient.
model_run_id	int	FK	Yes	-> model_run; null when no probability.
temperature_f	decimal(4,1)		Yes	Celsius values converted.
heart_rate, resp_rate, o2_sat	int		Yes	Out-of-range values set to missing.
sbp, dbp	int		Yes	dbp 879 set to missing.
pain_score	tinyint		Yes	0 to 10 only; text and values above 10 removed.
acuity_esi	tinyint		Yes	1 to 5 as given at triage; 15 missing.
chief_complaint	varchar(200)		No	
shock_index, map	decimal(6,3)		Yes	Heart rate / SBP; (SBP + 2 x DBP) / 3.
risk_score	tinyint		Yes	Null when Unscored.
risk_tier	varchar(9)		No	High, Medium, Low or Unscored.
sort_order	int		No	High, Unscored, Medium, Low, then score descending.
outcome_probability	decimal(5,4)		Yes	Random Forest output.

dw.fact_admission

Column	Type	Key	Null	Notes
hadm_id	int	PK	No	
patient_key	int	FK	No	
admission_type_key	smallint	FK	No	
admit_time, discharge_time	datetime2		No	Shifted dates.
ed_reg_time, ed_out_time	datetime2		Yes	Present for 182 admissions.
ed_reg_hour	tinyint		Yes	0 to 23; used for the hourly chart.
los_days	decimal(6,2)		No	Discharge minus admit.
discharge_location	varchar(40)		Yes	Missing for 42 admissions.
in_hospital_death	bit		No	From hospital_expire_flag (15 rows = 1).

dw.fact_lab_result

Column	Type	Key	Null	Notes
labevent_id	int	PK	No	
patient_key	int	FK	No	
hadm_id	int	FK	Yes	Null for 28,420 results with no admission.
lab_test_key	int	FK	No	
charttime	datetime2		No	Used by the Patient page time axis.
value_num	float		Yes	Numeric result; null for text results.
value_text	varchar(200)		Yes	NEG, HOLD, ...
unit	varchar(20)		Yes	Kept per result: 4 tests have more than one unit.
ref_low, ref_high	float		Yes	Normal range at the time of the result.
is_abnormal	bit		No	From the source flag, and re-checked against the range.

dw.model_run, dw.etl_run_log, dw.data_quality_log

Table	Columns
model_run	model_run_id (PK), trained_at, algorithm, parameters, accuracy, auc.
etl_run_log	run_id (PK), step_name, started, finished, rows_in, rows_out, status.
data_quality_log	log_id (PK), run_id (FK), table_name, column_name, key_value, old_value, new_value, rule.

2.5 Physical Schema (SQL Server)

The statements below create the warehouse. Data types are chosen for the data seen in the files; names use lower case with underscores.

```
CREATE DATABASE Mimic;  GO
USE Mimic;  GO
CREATE SCHEMA ed;  GO      -- raw triage
CREATE SCHEMA hosp;  GO    -- raw hospital tables
CREATE SCHEMA dw;  GO      -- processed results

CREATE TABLE dw.dim_patient (
    patient_key        INT IDENTITY(1,1) PRIMARY KEY,
    masked_id          VARCHAR(8)  NOT NULL,
    subject_id_hash    CHAR(64)    NOT NULL UNIQUE,
    gender             CHAR(1)     NOT NULL CHECK (gender IN ('M','F')),
    age                INT         NOT NULL,
    anchor_year_group  VARCHAR(12) NOT NULL );

CREATE TABLE dw.dim_admission_type (
    admission_type_key SMALLINT IDENTITY(1,1) PRIMARY KEY,
    admission_type     VARCHAR(40) NOT NULL UNIQUE,
    type_group         VARCHAR(20) NOT NULL );

CREATE TABLE dw.dim_lab_test (
    lab_test_key INT IDENTITY(1,1) PRIMARY KEY,
    itemid       INT          NOT NULL UNIQUE,
    label        VARCHAR(100) NOT NULL,
    fluid        VARCHAR(30)  NOT NULL,
    category     VARCHAR(20)  NOT NULL );

CREATE TABLE dw.model_run (
    model_run_id INT IDENTITY(1,1) PRIMARY KEY,
    trained_at   DATETIME2 NOT NULL,  algorithm VARCHAR(40) NOT NULL,
    parameters   VARCHAR(400) NULL,   accuracy  DECIMAL(5,4) NULL,  auc DECIMAL(5,4) NULL );

CREATE TABLE dw.fact_triage_scored (
    stay_id        INT PRIMARY KEY,
    patient_key    INT NOT NULL REFERENCES dw.dim_patient(patient_key),
    model_run_id   INT NULL     REFERENCES dw.model_run(model_run_id),
    temperature_f  DECIMAL(4,1) NULL,  heart_rate INT NULL,  resp_rate INT NULL,  o2_sat INT NULL,
    sbp INT NULL,  dbp INT NULL,
    pain_score     TINYINT NULL CHECK (pain_score BETWEEN 0 AND 10),
    acuity_esi     TINYINT NULL CHECK (acuity_esi BETWEEN 1 AND 5),
    chief_complaint VARCHAR(200) NOT NULL,
    shock_index    DECIMAL(6,3) NULL,  map DECIMAL(6,2) NULL,
    risk_score     TINYINT NULL,
    risk_tier      VARCHAR(9) NOT NULL CHECK (risk_tier IN ('High','Medium','Low','Unscored')),
    sort_order     INT NOT NULL,
    outcome_probability DECIMAL(5,4) NULL );

CREATE TABLE dw.fact_admission (
    hadm_id        INT PRIMARY KEY,
    patient_key    INT NOT NULL REFERENCES dw.dim_patient(patient_key),
    admission_type_key SMALLINT NOT NULL REFERENCES dw.dim_admission_type(admission_type_key),
    admit_time DATETIME2 NOT NULL,  discharge_time DATETIME2 NOT NULL,
    ed_reg_time DATETIME2 NULL,  ed_out_time DATETIME2 NULL,  ed_reg_hour TINYINT NULL,
    los_days DECIMAL(6,2) NOT NULL,  discharge_location VARCHAR(40) NULL,  in_hospital_death BIT NOT NULL );

CREATE TABLE dw.fact_lab_result (
    labevent_id INT PRIMARY KEY,
    patient_key INT NOT NULL REFERENCES dw.dim_patient(patient_key),
    hadm_id     INT NULL     REFERENCES dw.fact_admission(hadm_id),
    lab_test_key INT NOT NULL REFERENCES dw.dim_lab_test(lab_test_key),
    charttime DATETIME2 NOT NULL,  value_num FLOAT NULL,  value_text VARCHAR(200) NULL,
    unit VARCHAR(20) NULL,  ref_low FLOAT NULL,  ref_high FLOAT NULL,  is_abnormal BIT NOT NULL );

CREATE TABLE dw.etl_run_log (
    run_id INT IDENTITY(1,1) PRIMARY KEY, step_name VARCHAR(40) NOT NULL,
    started DATETIME2 NOT NULL, finished DATETIME2 NULL, rows_in INT NULL, rows_out INT NULL, status VARCHAR(10) NOT NULL );
CREATE TABLE dw.data_quality_log (
    log_id INT IDENTITY(1,1) PRIMARY KEY, run_id INT NOT NULL REFERENCES dw.etl_run_log(run_id),
    table_name VARCHAR(60) NOT NULL, column_name VARCHAR(60) NOT NULL, key_value VARCHAR(40) NULL,
```

```
    old_value VARCHAR(60) NULL, new_value VARCHAR(60) NULL, rule VARCHAR(80) NOT NULL );
```

Indexes

Index	On	Why
IX_lab_patient_time	fact_lab_result (patient_key, lab_test_key, charttime)	The Patient page reads one patient's results for one test over time.
IX_lab_abnormal	fact_lab_result (patient_key) INCLUDE (lab_test_key, charttime, value_num) WHERE is_abnormal = 1	Latest abnormal result per test; filtered index keeps it small.
IX_lab_admission	fact_lab_result (hadm_id)	Join from admissions to their lab results.
IX_triage_queue	fact_triage_scored (sort_order)	The Triage page lists records in priority order.
IX_adm_patient	fact_admission (patient_key, admit_time)	Patient profile and admissions list.

```
CREATE VIEW dw.vw_triage_queue AS
SELECT t.stay_id, p.masked_id, t.chief_complaint, t.temperature_f, t.heart_rate, t.resp_rate, t.o2_sat,
       t.sbp, t.dbp, t.pain_score, t.acuity_esi, t.risk_score, t.risk_tier, t.sort_order, t.outcome_probability
FROM dw.fact_triage_scored t JOIN dw.dim_patient p ON p.patient_key = t.patient_key;
GO
CREATE VIEW dw.vw_patient_timeline AS
SELECT p.masked_id, p.gender, p.age, a.hadm_id, at.admission_type, a.admit_time, a.discharge_time, a.los_days,
       a.discharge_location, l.charttime, lt.label AS lab_name, l.value_num, l.unit, l.ref_low, l.ref_high, l.is_abnormal
FROM dw.dim_patient p
JOIN dw.fact_admission a ON a.patient_key = p.patient_key
JOIN dw.dim_admission_type at ON at.admission_type_key = a.admission_type_key
LEFT JOIN dw.fact_lab_result l ON l.hadm_id = a.hadm_id
LEFT JOIN dw.dim_lab_test lt ON lt.lab_test_key = l.lab_test_key;
GO
CREATE VIEW dw.vw_exec_kpis AS
SELECT a.hadm_id, at.admission_type, a.ed_reg_time, a.ed_reg_hour, DATENAME(weekday, a.ed_reg_time) AS ed_weekday,
       a.los_days, a.discharge_location, a.in_hospital_death
FROM dw.fact_admission a JOIN dw.dim_admission_type at ON at.admission_type_key = a.admission_type_key;
```

2.6 Normalization and Modelling Decisions

The raw tables are kept as supplied. The warehouse is designed on purpose as a **star schema**: facts are in third normal form with respect to their own key, and dimensions hold descriptive text once. The table below records what was split out and what was kept together.

Evidence in the data	Decision	Normal form / reason
admission_type has 9 repeated text values in 275 rows.	Moved to dim_admission_type with a key.	3NF: description depends on the type, not on the admission.
Lab test name, fluid and category repeat on all 107,727 lab rows.	Moved to dim_lab_test (itemid).	3NF: label, fluid, category depend only on itemid.
Gender, age and year group repeat for every record of a patient.	Held once in dim_patient.	2NF: depends on the patient, not on the stay or lab.
The same lab test is reported in 4 different units.	Unit stays on the lab result row.	Unit is not fixed by itemid, so moving it would lose information.
Reference range changes with patient and time.	ref_low, ref_high stay on the lab result row.	They depend on the result, not only on the test.
The lab flag has one value, 'abnormal', or is empty.	Stored as bit is_abnormal.	Atomic value (1NF); easier to count.
Many labs have no admission.	hadm_id is optional (NULL) in fact_lab_result.	Keeps all 107,727 results; no fake admissions created.
Triage cannot be linked to an admission.	fact_triage_scored links to the patient only.	Avoids a wrong relationship.
Real subject_id must stay private.	dim_patient stores a hash and a masked ID; no raw ID in dw.	Privacy; the pipeline keeps the real ID only in memory (private attribute).
Dimensions repeat descriptive text (gender, year group).	Left denormalized in the dimension.	Normal star-schema practice: fewer joins for the report.

2.7 Source-to-Warehouse Mapping and Cleaning Rules

Source	Target	Rule
ed.triage.temperature	fact_triage_scored.temperature_f	Values below 45 are Celsius: multiply by 9/5 and add 32 (36.5 gives 97.7). Log the change.
ed.triage.dbp	dbp	Outside the allowed range in the settings file (for example 879) becomes missing. Log the change.
ed.triage.heartrate, resprate, o2sat, sbp	heart_rate, resp_rate, o2_sat, sbp	Same range check from the settings file.
ed.triage.pain	pain_score	Keep whole numbers 0 to 10; text (for example 'unable') and values such as 13 become missing.
ed.triage.acuity	acuity_esi	Whole number 1 to 5; keep missing as missing.
ed.triage (5 vitals)	risk_score, risk_tier	Points per vital sign from the bands; fewer than 3 recorded means tier Unscored and score null.
hosp.patients.subject_id	dim_patient	Hash for the key; mask all but the last 3 digits.
hosp.admissions.hospital_expire_flag	in_hospital_death	1 becomes true.
hosp.admissions.language	(not in the report)	'?' becomes Unknown if the column is used later.
hosp.admissions.admittime, dischtime	los_days	Days between admit and discharge.
hosp.admissions.edregtime	ed_reg_time, ed_reg_hour	Hour of the day, 0 to 23; null if no ED time.
hosp.labevents.flag	is_abnormal	'abnormal' becomes 1; empty becomes 0; also set to 1 if the value is outside the range.
hosp.labevents.value, valuenum	value_text, value_num	Numeric result goes to value_num; text results stay in value_text.
hosp.d_labitems	dim_lab_test	Missing label becomes 'Unknown test '; fluid codes 'Q', 'l', 'Fluid' become Other.

2.8 Example Queries and Results

The queries below are written for SQL Server. The results are taken from the project files and can be used to check the warehouse after loading.

Q1. Admissions and in-hospital deaths by admission type

```
SELECT at.admission_type, COUNT(*) AS admissions, SUM(CAST(a.in_hospital_death AS INT)) AS deaths,
       ROUND(100.0 * SUM(CAST(a.in_hospital_death AS INT)) / COUNT(*), 1) AS death_rate_pct
FROM dw.fact_admission a JOIN dw.dim_admission_type at ON at.admission_type_key = a.admission_type_key
GROUP BY at.admission_type ORDER BY admissions DESC;
```

Admission type	Admissions	Deaths	Death rate %
EW EMER.	104	6	5.8
OBSERVATION ADMIT	45	3	6.7
URGENT	38	5	13.2
EU OBSERVATION	30	0	0.0
SURGICAL SAME DAY ADMISSION	18	0	0.0
DIRECT EMER.	15	1	6.7
ELECTIVE	13	0	0.0
DIRECT OBSERVATION	7	0	0.0
AMBULATORY OBSERVATION	5	0	0.0

Q2. Share of abnormal lab results by category

```
SELECT lt.category, COUNT(*) AS results, SUM(CAST(l.is_abnormal AS INT)) AS abnormal,
       ROUND(100.0 * SUM(CAST(l.is_abnormal AS INT)) / COUNT(*), 1) AS abnormal_pct
FROM dw.fact_lab_result l JOIN dw.dim_lab_test lt ON lt.lab_test_key = l.lab_test_key GROUP BY lt.category;
```

Category	Results	Abnormal	Abnormal %
Blood Gas	9,329	3,058	32.8
Chemistry	47,125	13,374	28.4
Hematology	51,273	23,843	46.5

Q3. Records in each risk tier (check for the Triage page cards)

```
SELECT risk_tier, COUNT(*) AS records FROM dw.fact_triage_scored GROUP BY risk_tier;
```

Risk tier	Records
High	3
Medium	17
Low	179
Unscored	23
Total	222

Q4. Latest abnormal result per test for one patient

```
SELECT lab_name, value_num, unit, ref_low, ref_high, charttime
FROM ( SELECT lt.label AS lab_name, l.value_num, l.unit, l.ref_low, l.ref_high, l.charttime,
              ROW_NUMBER() OVER (PARTITION BY l.lab_test_key ORDER BY l.charttime DESC) AS rn
       FROM dw.fact_lab_result l JOIN dw.dim_lab_test lt ON lt.lab_test_key = l.lab_test_key
       JOIN dw.dim_patient p ON p.patient_key = l.patient_key
       WHERE p.masked_id = '***810' AND l.is_abnormal = 1 ) x
WHERE rn = 1;
```

3. Data Flow & System Behavior

This section shows how data moves through the system and how the parts behave over time. Data flow diagrams show **what data goes where**. Sequence and activity diagrams show **the order of steps**. State diagrams show **the stages a record or a run passes through**. The class diagram shows **the Python classes** that implement the pipeline.

3.1 Data Flow Diagrams

Context diagram (level 0 of detail)

The whole system is one process. Four people use it, and one external source feeds it.

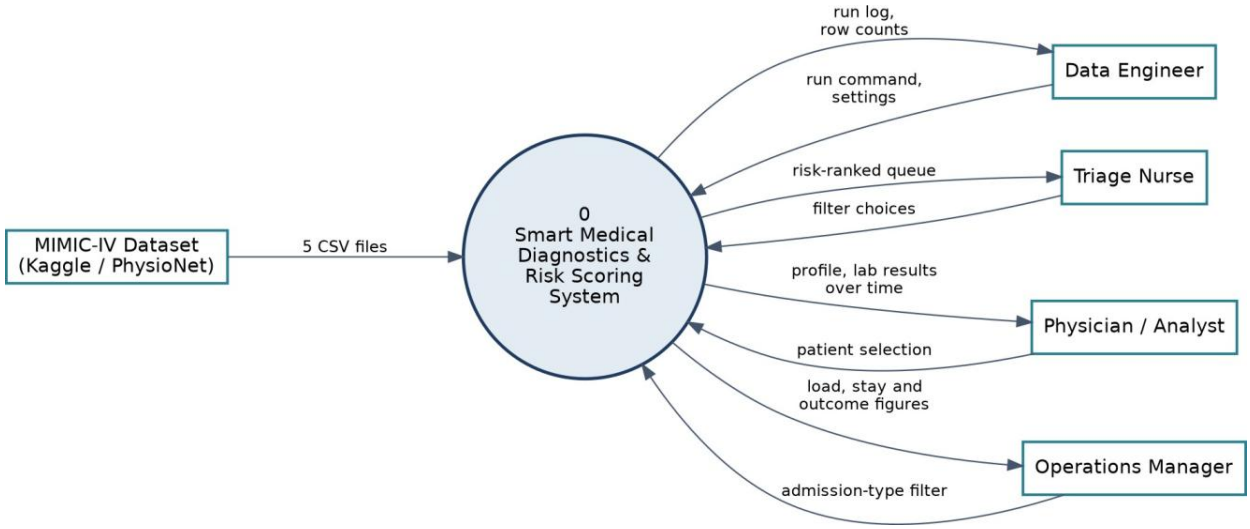


Figure 3.1. DFD context diagram.

Flow	From	To	Content
5 CSV files	MIMIC-IV dataset	System	ed/triage, hosp/patients, admissions, labevents, d_labitems.
Run command, settings	Data engineer	System	Start of a run; paths, ranges, score bands.
Run log, row counts	System	Data engineer	Every change made, steps completed, counts compared.
Filter choices	Triage nurse	System	Acuity or tier selected on the Triage page.
Risk-ranked queue	System	Triage nurse	Records ordered by tier and score.
Patient selection	Physician / analyst	System	Row chosen for drill-through; lab test chosen.
Profile, lab results over time	System	Physician / analyst	Patient page content.
Admission-type filter	Operations manager	System	Selection on the Executive page.
Load, stay and outcome figures	System	Operations manager	Cards and charts on the Executive page.

Level 0 DFD: the seven main processes

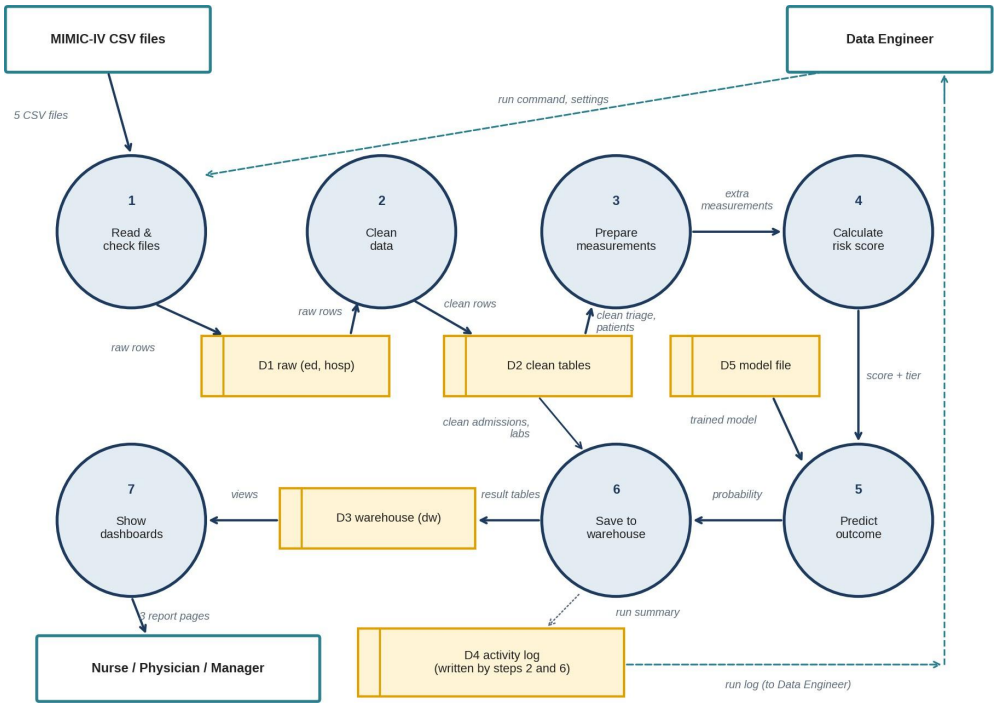


Figure 3.2. Level 0 DFD. Circles are processes, open rectangles are data stores, boxes are external entities.

Process	Input	Output	Data stores used
1 Read and check files	5 CSV files	Raw tables	D1 raw (ed, hosp)
2 Clean data	Raw tables, valid ranges	Clean tables, change lines	D1, D2 clean tables, D4 activity log
3 Prepare measurements	Clean triage and patients	Shock index, mean arterial pressure	D2
4 Calculate risk score	Vital signs, score bands	Score and tier, or Unscored	D2, D3
5 Predict outcome	Measurements, score	Probability for each record	D5 model file, D3
6 Save to warehouse	Clean tables, scores, probabilities	dw tables, run summary	D2, D3 warehouse (dw), D4
7 Show dashboards	Views in dw	Three report pages	D3

Level 1 DFDs for the two most important processes

Process 2 (cleaning) and process 4 (scoring) hold the rules that most affect the result, so they are shown in more detail.

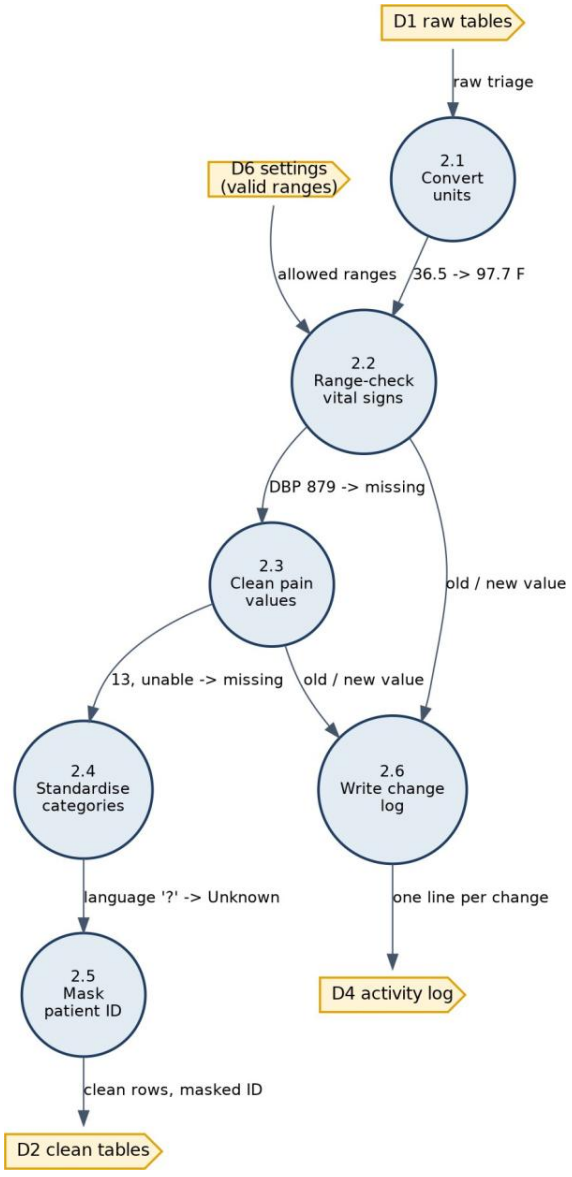


Figure 3.3. Level 1: process 2, clean data.

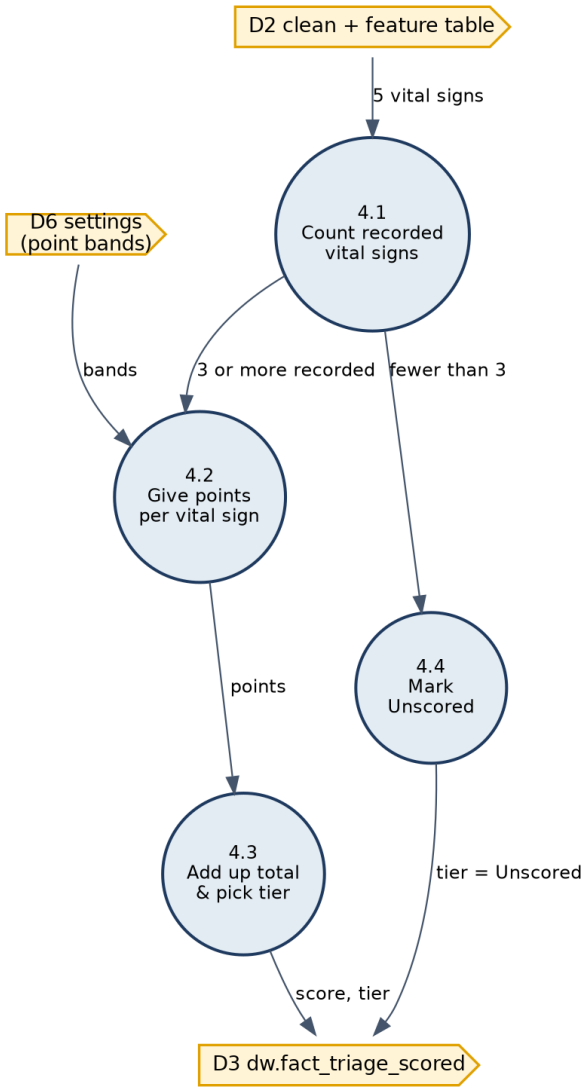


Figure 3.4. Level 1: process 4, calculate risk score.

3.2 Sequence Diagrams

Each diagram shows one scenario from start to finish. Vertical lines are the parts involved, arrows are messages in time order, and the amber frame marks a choice.

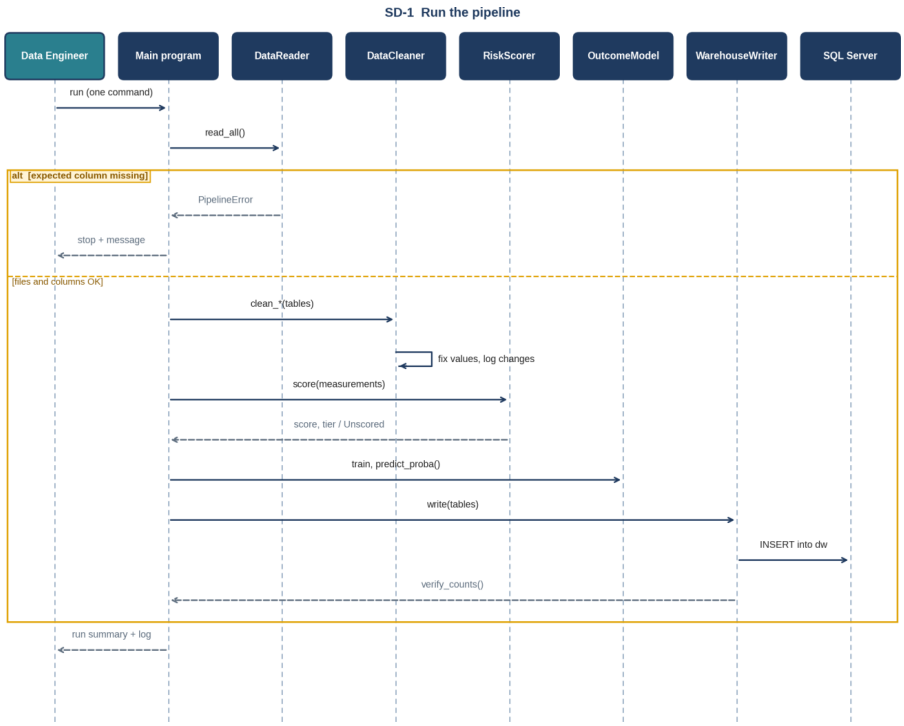


Figure 3.5. SD-1: running the pipeline. If a column is missing the run stops and nothing is written.

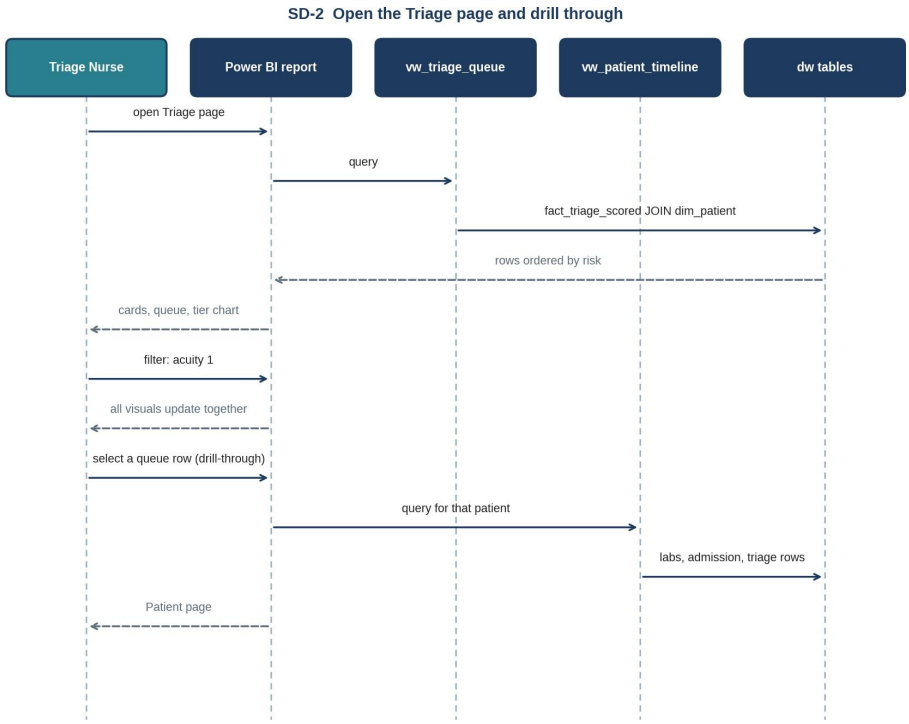


Figure 3.6. SD-2: a nurse opens the Triage page, filters and drills through to the Patient page.

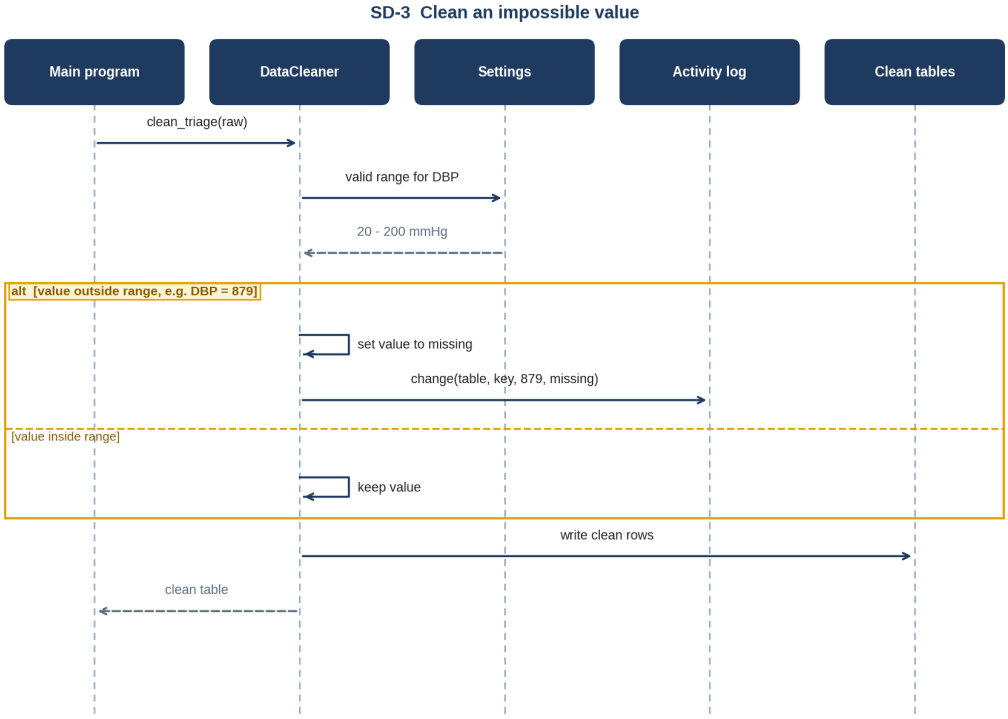


Figure 3.7. SD-3: cleaning an impossible value (diastolic pressure 879) and logging the change.

3.3 Activity Diagrams

The first diagram shows who does what during a pipeline run. The second shows how a nurse and a physician use the report.

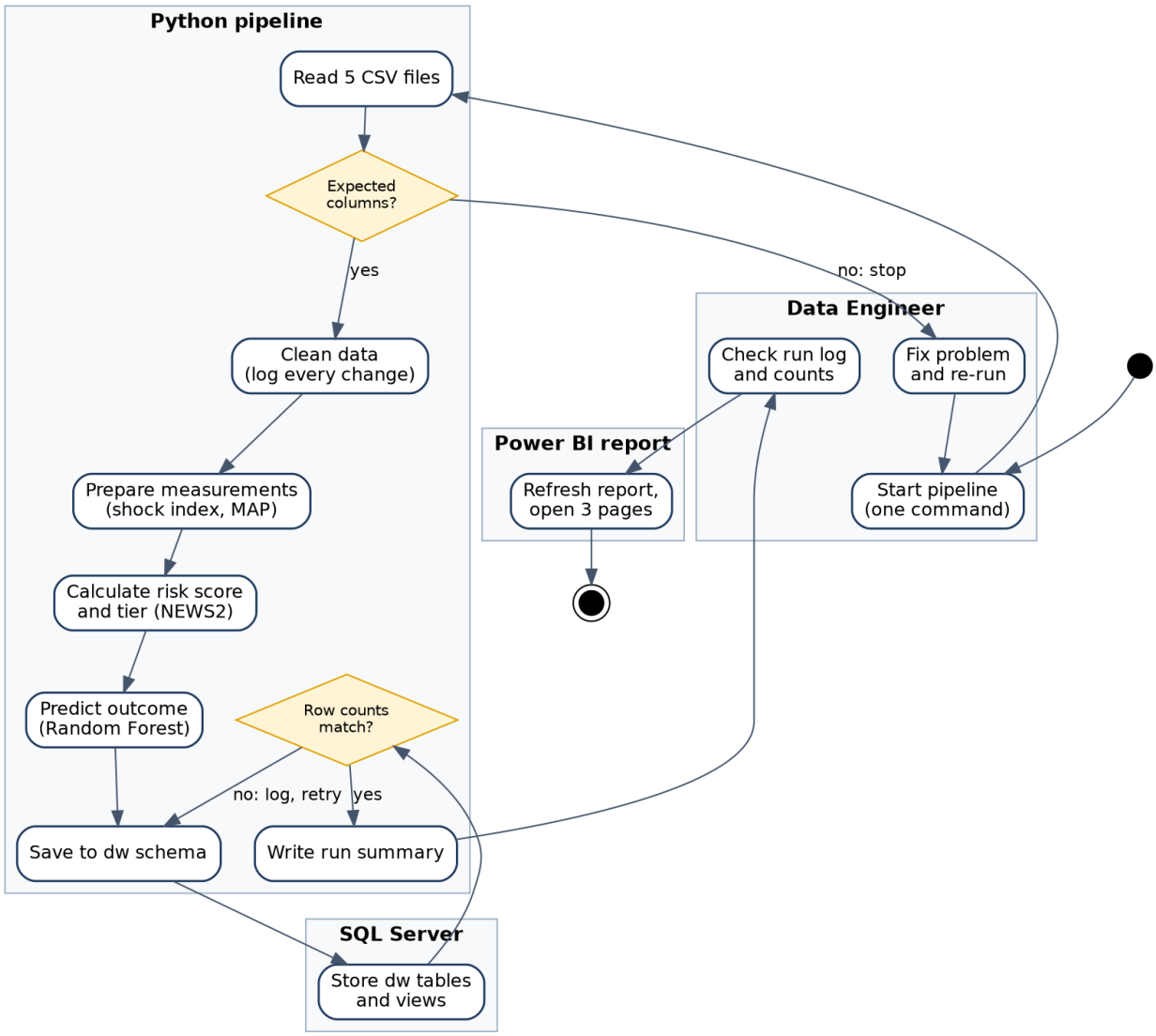


Figure 3.8. Activity diagram: a pipeline run, with swimlanes for engineer, pipeline, SQL Server and Power BI.

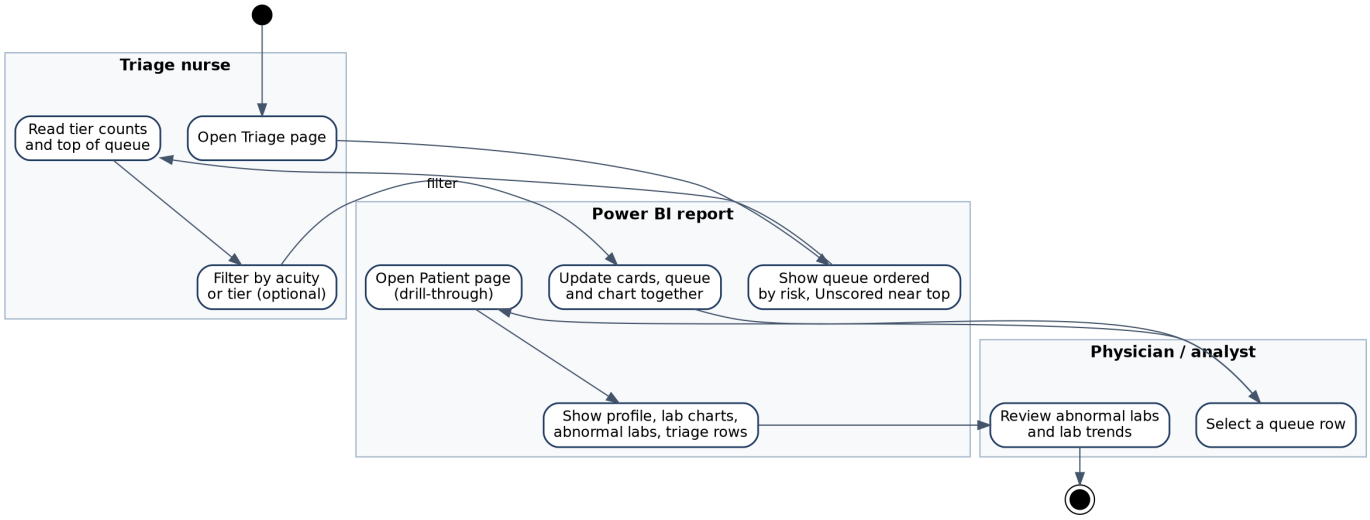


Figure 3.9. Activity diagram: using the Triage and Patient pages.

3.4 State Diagrams

State diagrams show the stages of three things: a triage record, a pipeline run and the model. A record can only be in one state at a time.

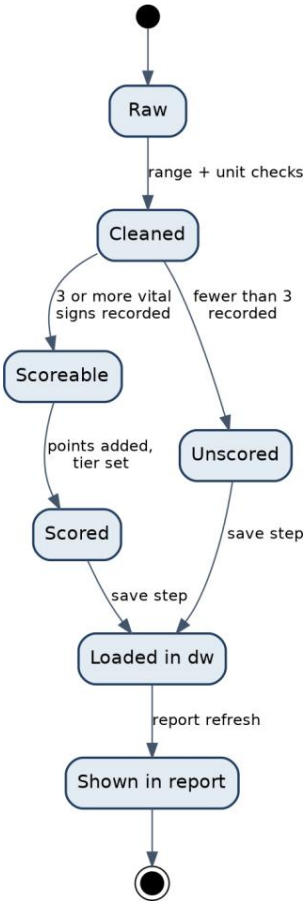


Figure 3.10. States of a triage record.

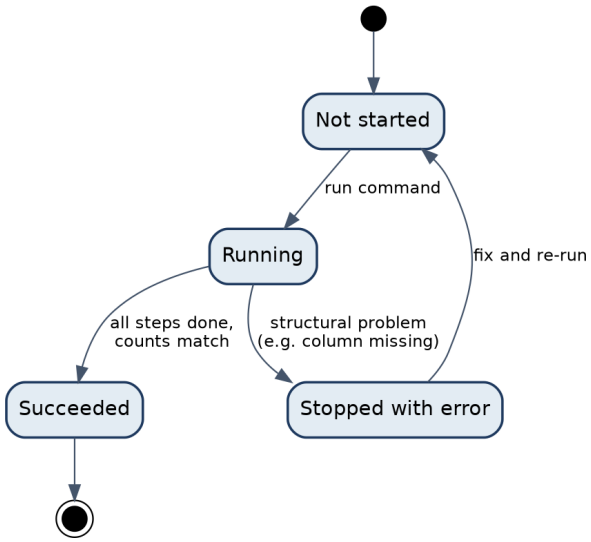


Figure 3.11. States of a pipeline run.

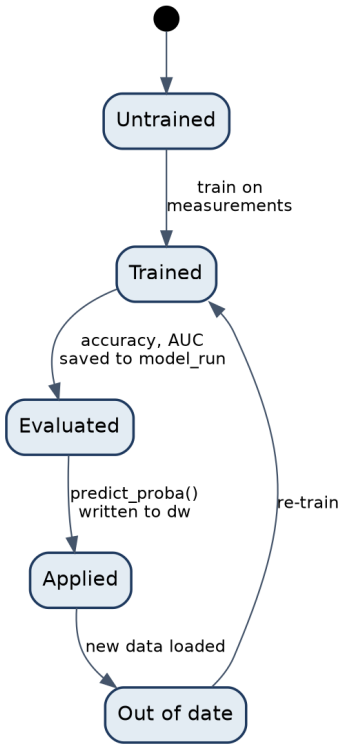


Figure 3.12. States of the model.

State	Meaning
Raw	As read from the CSV file. May hold impossible values.
Cleaned	Units fixed, impossible values set to missing, changes logged.
Scoreable / Unscored	Scoreable: at least 3 of 5 vital signs present. Unscored: fewer than 3; shown as grey, near the top.
Scored	Points added and tier set (High, Medium or Low).
Loaded in dw	Saved in dw.fact_triage_scored.
Shown in report	Visible on the Triage page after a refresh.
Stopped with error	The run met a structural problem such as a missing column and wrote nothing.
Out of date (model)	New data was loaded after the last training; the model should be re-trained.

3.5 Class Diagram

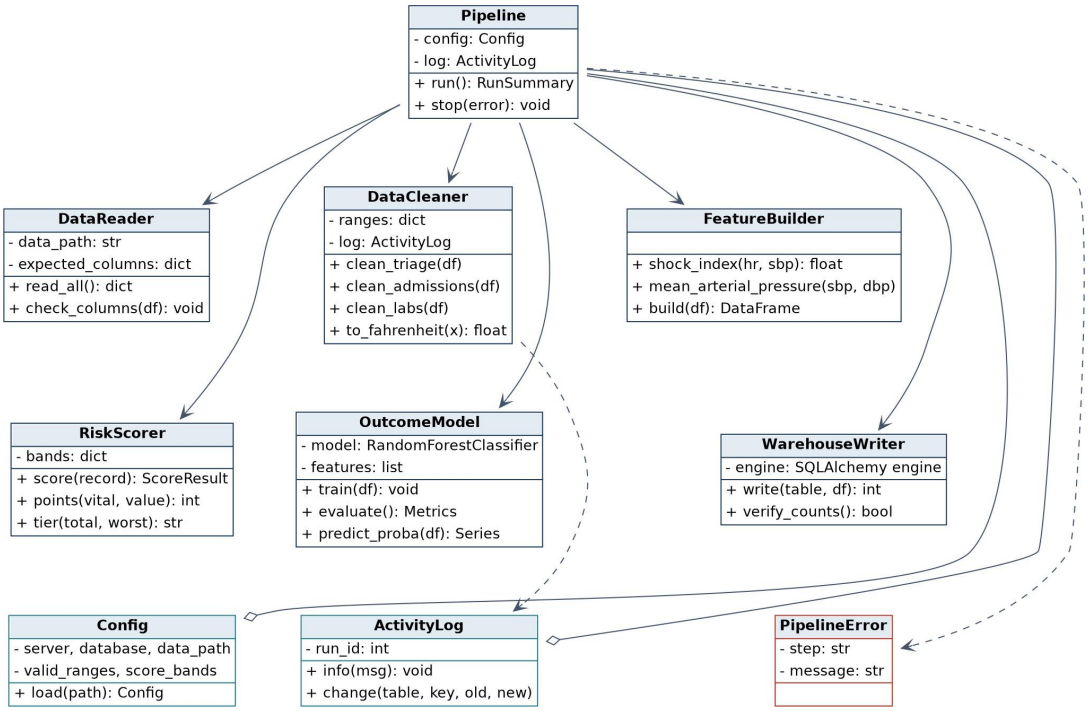


Figure 3.13. Class diagram, pipeline classes. Filled diamonds show ownership; dashed arrows show use.

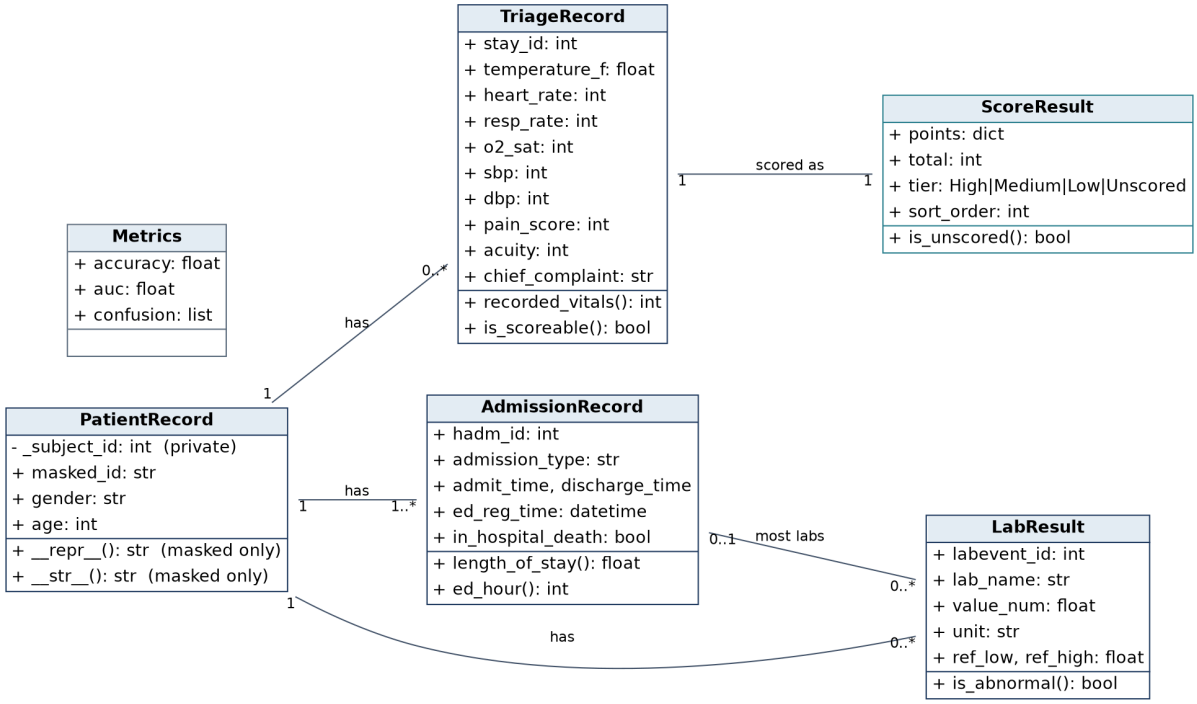


Figure 3.14. Class diagram, record classes. PatientRecord keeps the real ID private.

Class	Responsibility	Main methods
Pipeline	Runs the steps in order; stops and reports on a structural error.	run(), stop(error)
DataReader	Loads the five CSV files and confirms the expected columns.	read_all(), check_columns()
DataCleaner	Fixes units and impossible values; logs each change.	clean_triage(), clean_admissions(), clean_labs(), to_fahrenheit()
FeatureBuilder	Works out shock index and mean arterial pressure.	shock_index(), mean_arterial_pressure(), build()
RiskScorer	Gives points per vital sign and sets the tier.	score(), points(), tier()
OutcomeModel	Trains the Random Forest, evaluates it, gives probabilities.	train(), evaluate(), predict_proba()
WarehouseWriter	Writes tables to dw and compares row counts.	write(), verify_counts()
Config	Holds server name, paths, valid ranges, score bands.	load()
ActivityLog	Records every step and every change to the data.	info(), change()
PipelineError	Error with the step name and a clear message.	-
PatientRecord	Holds the real subject_id privately; prints only the masked ID.	__repr__(), __str__() show the masked ID
TriageRecord, ScoreResult	One triage stay and its score, tier and sort order.	recorded_vitals(), is_scoreable()
AdmissionRecord, LabResult	One admission and one lab result with derived values.	length_of_stay(), is_abnormal()

Design notes

- **Single responsibility.** Each class does one thing, so each can be tested on its own (section 6.2).
- **Settings, not constants.** Ranges and score bands are read from the Config object, so changing a band does not change code.
- **Private ID.** PatientRecord stores the subject ID in a private attribute; printing or logging a patient shows only the masked ID (test U1).
- **Fail early.** The reader raises PipelineError when a column is missing; the main program catches it and stops before anything is written.

4. UI/UX Design & Prototyping

The front end is a Power BI report with three pages: Triage, Patient and Executive. There is no web or mobile interface. The screens are designed around the data that exists in the five source tables, which sets three limits.

- **ed.triage** has no timestamp and no key to admissions. The Triage page therefore has no waiting-time column and orders records by risk tier and score.
- Triage vitals are one snapshot per record. Time-based charts on the Patient page use lab results (**hosp.labevents**), which carry *charttime*.
- ED registration time exists only in **hosp.admissions** (*edregtime*), and only for admissions that came through the emergency department. Arrival-pattern charts on the Executive page cover those admissions only.

Source table	Key	Content used by the report
ed.triage	stay_id	Vitals, pain, acuity (ESI), chief complaint. Triage and Patient pages.
hosp.patients	subject_id	Gender, anchor_age. Patient page.
hosp.admissions	hadm_id	Admission type, admit and discharge time, discharge location, death flag, ED registration and exit time. Patient and Executive pages.
hosp.labevents	labevent_id	Lab value, unit, reference range, abnormal flag, charttime. Patient page.
hosp.d_labitems	itemid	Lab name, fluid, category. Patient page.

1. Wireframes & Mockups

1. Report structure



Page tabs sit at the bottom-left of every page. Drill-through goes from a queue row to that patient's page.

Figure 4.1. Page flow of the report.

Page	User	Purpose	Main visuals	Fields
Triage	Triage nurse	Order records by risk and show which have no vitals.	Four tier KPI cards, priority queue table, records-by-tier chart, two slicers.	ed.triage: stay_id, subject_id, vitals, pain, acuity, chiefcomplaint. Calculated: risk_score, risk_tier.
Patient	Physician, analyst	Show one patient's profile, lab history and triage records.	Profile strip, two lab line charts, abnormal lab table, triage table, two slicers.	patients: gender, anchor_age. admissions: type, times, discharge_location. labevents: charttime, valuenum, valueuom, ref_range_lower/upper, flag. d_labitems: label.
Executive	Operations manager	Show load, length of stay and outcomes.	Four KPI cards, hourly registration chart, stay-length chart, admission-type chart, one slicer.	admissions: edregtime, admittime, dischtime, admission_type, hospital_expire_flag.

Risk tiers on the Triage page follow one rule set (draft bands).

Tier	Colour	Rule
High	Red	Score of 7 or more.
Medium	Amber	Score of 5 or 6, or any single vital scoring 3.
Low	Green	Score of 0 to 4 and no single vital scoring 3.
Unscored	Slate	Fewer than 3 of the 5 scored vitals present (heart rate, respiratory rate, oxygen saturation, systolic pressure, temperature).

4.1.2 Wireframes

Low-fidelity layouts. Dashed boxes are placeholders; numbers refer to the notes under each figure.

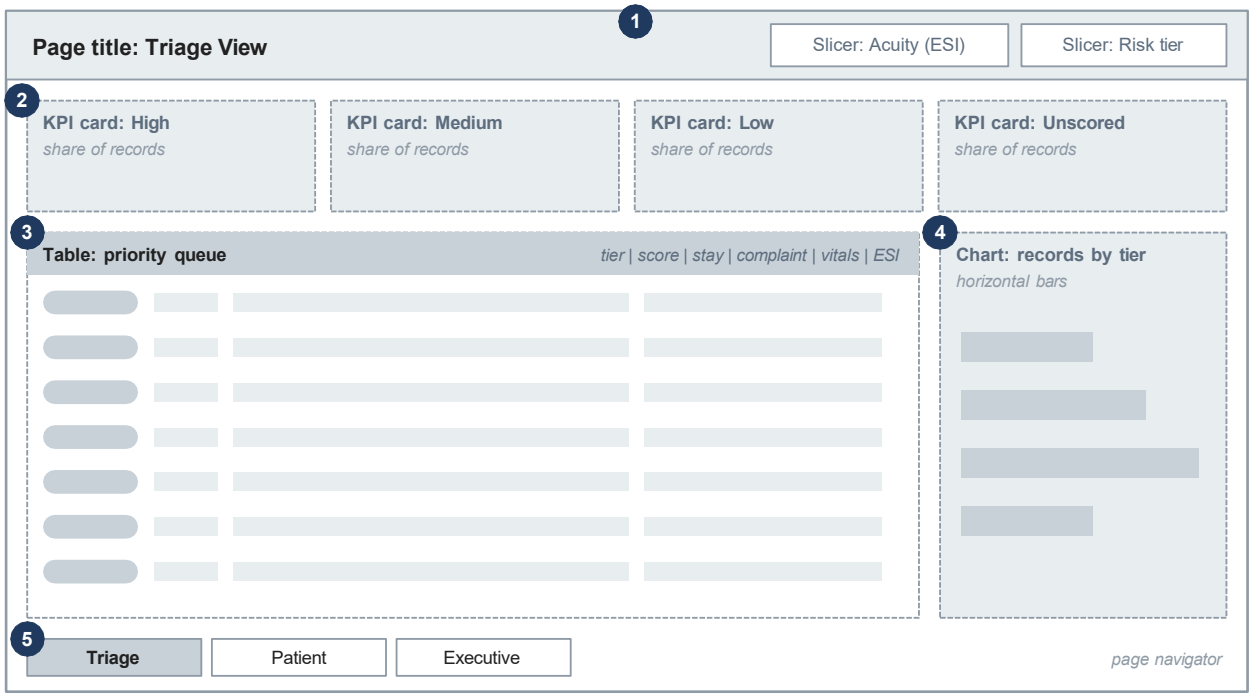


Figure 4.2. Wireframe, Triage page.

1. Slicers for ESI acuity and risk tier.
2. KPI cards: share of records in each tier.
3. Priority queue: tier, score, stay ID, chief complaint, vitals, ESI. Order: High, Unscored, Medium, Low; within a tier by score, highest first.
4. Bar chart of records by tier. Selecting a bar filters the queue.
5. Page navigator.

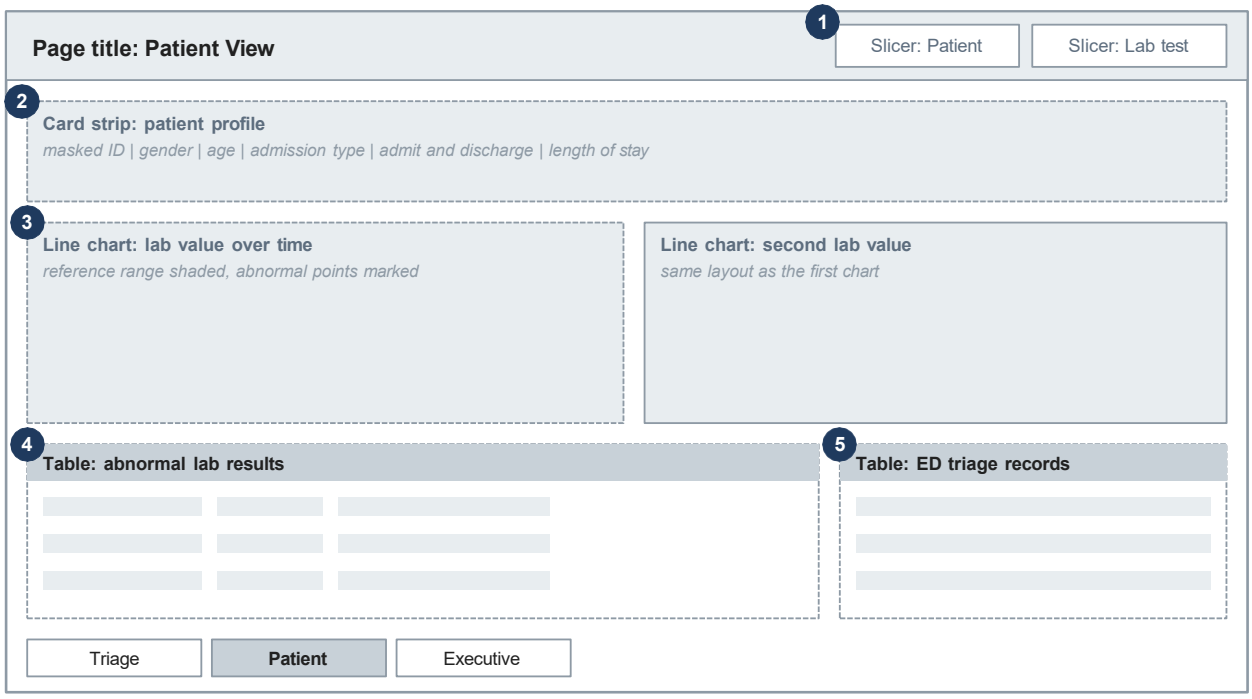


Figure 4.3. Wireframe, Patient page.

1. Slicers: patient (masked ID) and lab test.
2. Profile strip: gender, age, admission type, admit and discharge time, length of stay.
3. Two line charts of lab values over charttime. The shaded band is the reference range; abnormal results are marked.
4. Table of the latest abnormal result per lab test.
5. Table of the patient's ED triage records with their tier.

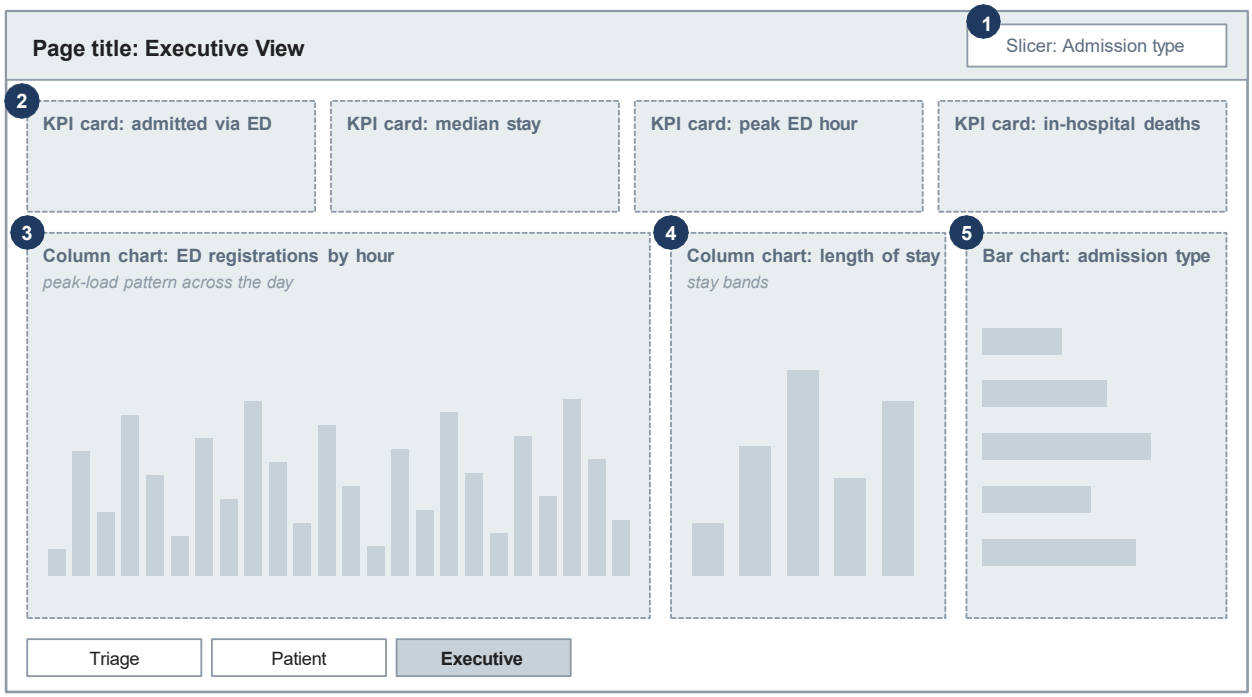


Figure 4.4. Wireframe, Executive page.

1. Slicer for admission type.

2. KPI cards: share of admissions that came through the ED, median stay, peak ED hour, share of in-hospital deaths.
3. Column chart of ED registrations by hour of day.
4. Column chart of length of stay in bands.
5. Bar chart of admission type.

4.2 UI/UX Guidelines

Design principles

- **Priority first.** The most urgent information sits top-left and is readable without scrolling.
- **One layout.** All pages share the same title bar, slicer position, card style and page navigator.
- **Detail on demand.** Summary on the Triage and Executive pages; detail by drill-through to the Patient page.
- **Missing data is shown.** A blank vital appears as a dash and the record is tiered Unscored, never Low.
- **Few visuals per page.** Five to six, so each can be read at a glance.
- **Colour carries meaning.** Tier colours are used for risk only.

Colour scheme



Colour	Use
Navy	Page title bar, table headers, active page tab.
Teal	Chart series, secondary KPI accents.
Canvas	Page background behind white cards.
Red, amber, green, slate	Risk tiers High, Medium, Low, Unscored. Not used elsewhere.
Text	Values and labels.

Typography

Segoe UI, the Power BI default, on a 1280 x 720 canvas (16:9). Figures in this document are scaled down.

Element	Size	Weight
Page title	18 pt	Semibold
Visual title	12 pt	Semibold
KPI value	26 pt	Bold
Table text and axis labels	10 pt minimum	Regular
Footnotes	9 pt	Regular, italic

Accessibility

- Every tier flag carries a text label as well as a colour.
- Alt text on every visual; a meaningful title on each.
- Tab order follows reading order: slicers, KPI cards, main visual, secondary visuals.
- Pages are checked in greyscale to confirm that tiers can still be told apart.

Data presentation rules

- Units on screen: temperature in degrees F (source unit; values below 50 are treated as Celsius and converted), heart rate in bpm, respiratory rate in breaths per minute, oxygen saturation in %, blood pressure in mmHg.

5. System Deployment & Integration

1. Technology Stack

Part	Tool	What it does
Frontend	Power BI Desktop	Shows the three screens.
Backend	Python 3 with pandas, NumPy, scikit-learn, Seaborn, Matplotlib	Reads and cleans the data, calculates the risk score, trains the classification model and draws exploratory charts.
Backend	Python logging, pytest	Keeps a record of every change to the data and runs the tests.
Database	SQL Server (local), database Mimic	Stores the raw tables, in one section (schema) per source folder, and the processed results in a section called dw.
Source	Five CSV files	The data supplied from MIMIC-IV.

The data comes from five files:

File	Contains
ed / triage	Vital signs, urgency level and main complaint at triage.
hosp / patients	Gender and age.
hosp / admissions	Admission type, admit and discharge times, where the patient went afterwards, whether they died in hospital.
hosp / labevents	Lab results with units, normal ranges and an abnormal flag.
hosp / d_labitems	Names of the lab tests.

Patients are the link between them. One patient can have several admissions, triage records and lab results. Lab results point to the lab test names, and most also point to an admission. Triage records have no date or admission number, so they cannot be tied to a particular admission.

Database design

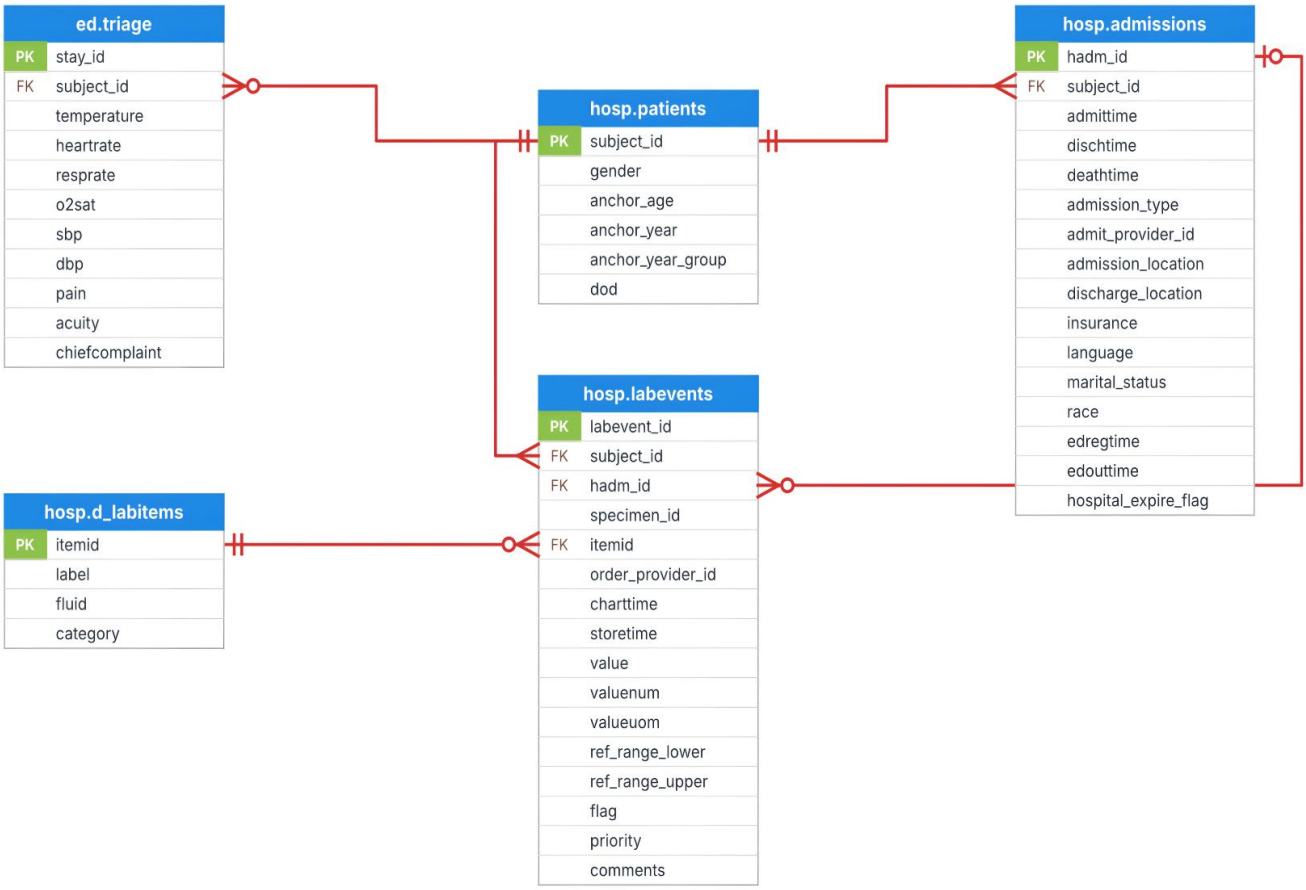


Figure 5.1. Entity-relationship diagram of the source tables.

Parent	Child	Key	Cardinality	Evidence in the data
hosp.patients	ed.triage	subject_id	1:M (Optional)	Not every patient has triage records; a patient can have several.
hosp.patients	hosp.admissions	subject_id	1:M (Mandatory)	Every patient has at least one admission and can have several.
hosp.patients	hosp.labevents	subject_id	1:M (Optional)	Every patient has lab results.
hosp.admissions	hosp.labevents	hadm_id	0..1 : M	hadm_id is empty on part of the lab rows, so the key is optional. Every non-empty value exists in admissions.
hosp.d_labitems	hosp.labevents	itemid	1:M (Optional)	Only part of the dictionary is used. Every itemid in labevents exists in d_labitems.
ed.triage	hosp.admissions	none	none	No shared key and no timestamp in triage. Related only through subject_id.

Primary keys (subject_id, hadm_id, stay_id, labevent_id, itemid) are unique in the files. The raw tables are loaded as text, so the database does not enforce keys on them; the integrity checks in the test plan do.

Processed tables in dw will use typed columns and primary keys.

.

5.2 Deployment Diagram

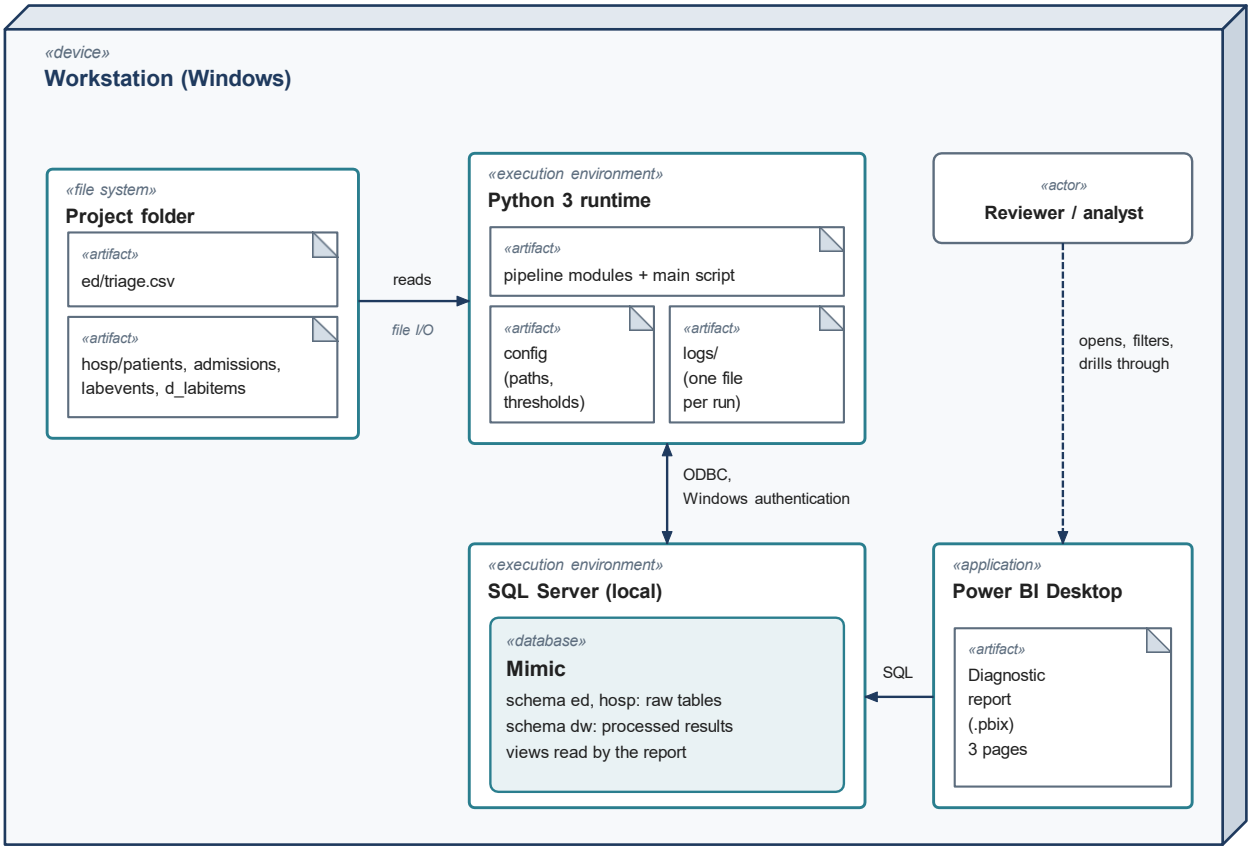


Figure 5.3. Deployment diagram (UML). All nodes run on one Windows workstation.

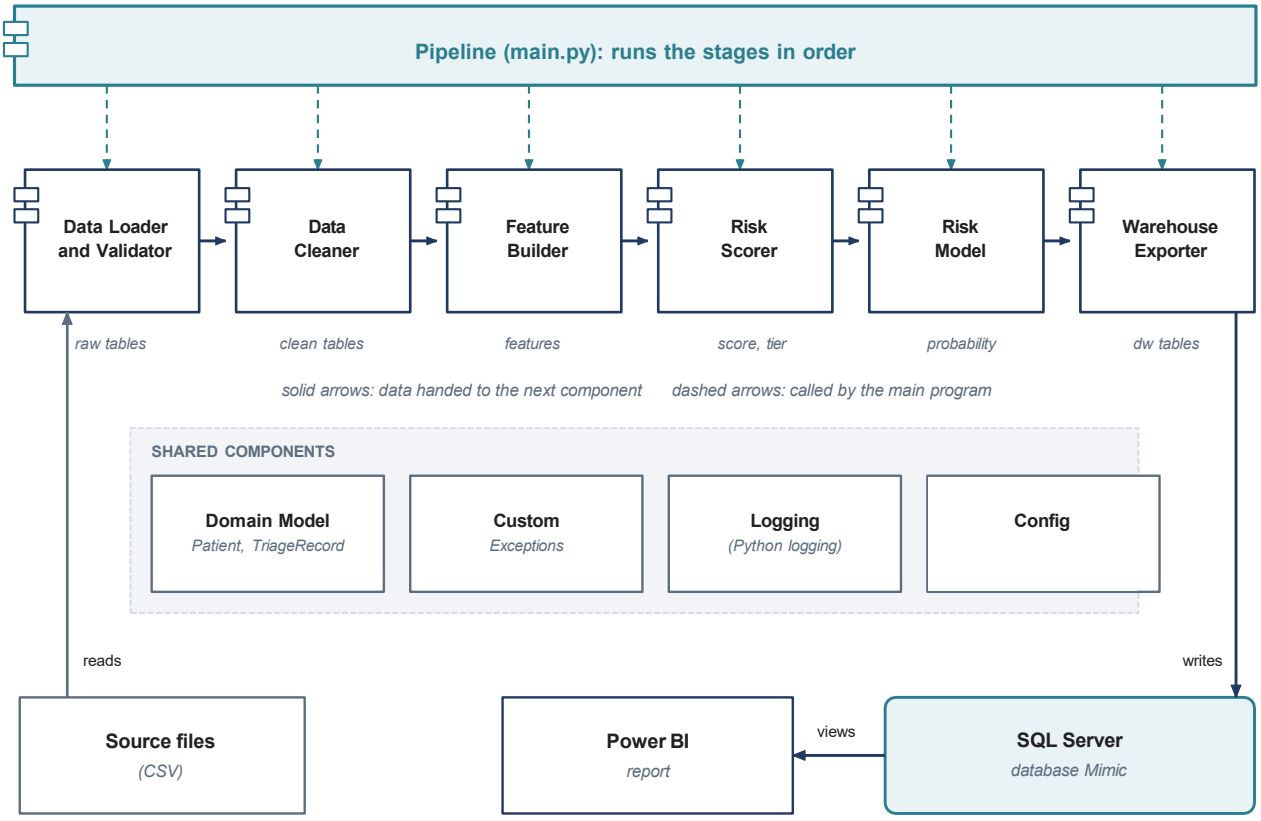
Element	Type	Description
Workstation (Windows)	Device	Single machine hosting every component. No network service is exposed.
Project folder	File system	ed/triage.csv and hosp/patients, admissions, labevents, d_labitems.
Python 3 runtime	Execution environment	Runs the pipeline modules and main script. Holds config (paths, server name, salt, thresholds) and writes one log file per run.
SQL Server (local)	Execution environment	Hosts database Mimic: raw schemas ed and hosp, result schema dw, and the views read by the report.

Element	Type	Description
Power BI Desktop	Application	Opens the .pbix report and queries the views.

Communication paths.

- Python reads the CSV files directly (file I/O).
- Python and SQL Server communicate over ODBC with Windows authentication, so no password is stored in the configuration.
- Power BI Desktop queries SQL Server through the SQL Server connector. Import or DirectQuery mode is still to be chosen.
- The report is refreshed by re-running the pipeline and then refreshing the data in Power BI.

5.3 Component Diagram



Interfaces between components are tables (pandas DataFrames in memory, SQL tables and views on disk).

Figure 5.4. Component diagram (UML). Solid arrows are data passed between components.

Component	Responsibility	Depends on	Hands over
Pipeline (main.py)	Calls each stage in order and stops on structural errors.	All components	Run summary
Data Loader and Validator	Reads the five CSV files and checks that expected columns exist.	Config, Exceptions, Logging	Raw tables
Data Cleaner	Applies plausibility and standardisation rules; logs every change.	Domain Model, Exceptions, Logging	Clean tables

6. Additional Deliverables

1. API Documentation

- The system has no API: no web service and no endpoints.
- Power BI reads three database views. **vw_triage_queue** feeds the Triage page (stay_id, masked patient key, chief complaint, vitals, pain, acuity, risk score, risk tier). **vw_patient_timeline** feeds the Patient page (patient, admission, lab results, triage records). **vw_exec_kpis** feeds the Executive page (ED registration time and hour, admission type, length of stay, discharge location, in-hospital death).
- Pipeline steps hand tables to each other: five CSV files, raw tables, clean tables, scored table, dw tables.

6.2 Testing & Validation

- **Unit tests:** the patient ID cannot be read directly; an impossible value (such as a diastolic pressure of 879) becomes missing and is logged; a temperature below 50 is converted from Celsius; invalid pain values become missing; known vitals give the expected score and tier; records with no vitals are Unscored.
- **Integration tests:** rows in equal rows out; triage to patients, labevents to admissions and labevents to d_labitems keys resolve; running the export twice gives the same counts; a full run from an empty dw schema finishes without manual steps.
- **Validation:** tier shares add up to 100% and report figures match SQL; High and Medium are more common at acuity 1 and 2 than at 3 and 4; a hand check of randomly chosen records.
- **User acceptance:** reviewers find the first record in the queue, filter by acuity, open a patient, find abnormal labs, find the busiest ED hour and read an Unscored record. A task passes when it is done without help.

6.3 Deployment Strategy

- **Environment:** one Windows workstation with local SQL Server, Python 3 and Power BI Desktop. Windows authentication.
- **Steps:** prepare the database (schemas ed, hosp, dw), load the CSV files into ed and hosp, run the pipeline, run the checks, refresh the report.
- **Scaling (not built):** indexes on subject_id, hadm_id and itemid and batch loading for larger extracts; Power BI service with scheduled refresh through a gateway for more users; a scheduled task to run the pipeline.